

第 16 章 Web应用系统分析与设计

Web 应用系统是指用户界面驻留在 Web 浏览器中的任意应用程序，它基于 Web 技术和 W3C 标准，通过一个用户界面（Web 浏览器或支持 Web 技术进行访问的可视化界面）提供 Web 特定的资源，如内容和服务等。随着新的 Web 平台、技术和标准不断演进，Web 应用的范围和复杂性千差万别，从小规模应用到大规模分布在 Internet、Intranet 和 Extranet 的企业应用，现在的 Web 应用系统提供了巨大的、各种各样的功能，并且有不同的特性和需求。Web 应用系统能使客户和商业公司具备在线商务处理、供应链集成、动态满足客户要求以及个性化服务等能力，系统的架构要求是健壮的、可伸缩的和可扩展的，可以提升性能并且能处理大量不可预知的商务交易负载。

本章在介绍 Web 应用系统的基础上，从 Web 应用架构设计、Web 应用开发技术框架、Web 应用系统开发过程及测试等方面全面介绍 Web 应用系统开发的相关知识。

16.1 Web 应用系统简介

16.1.1 Web 应用程序

Web 应用程序是在 Web 浏览器中运行的软件。企业在远程交换信息和提供服务时使用 Web 应用程序，以方便、安全地与客户联系。购物车、产品搜索和筛选、即时消息和社交媒体新闻源等最常见的网站功能均设计为 Web 应用程序，这些应用程序允许用户在不安装或配置软件的情况下访问复杂的功能。

Web 应用程序拥有多个优势，几乎所有大企业都将其作为向用户提供的产品和服务的一部分。以下是 Web 应用程序的一些最常见的优势。

(1) 易于访问。可以从所有 Web 浏览器及各种个人和企业设备访问 Web 应用程序。位于不同地点的团队可以通过基于订阅的 Web 应用程序访问共享文档、内容管理系统和其他业务服务。

(2) 开发高效。具体而言，Web 应用程序的开发过程相对简单，且对企业而言具有成本效益。小型团队可以实现较短的开发周期，使 Web 应用程序成为一种构建计算机程序的高效且经济的方法。此外，由于同一版本适用于所有现代浏览器和设备，因此开发人员无须为多个平台创建多个不同的迭代。

(3) 使用简单。Web 应用程序无须用户下载，易于访问，同时使得最终用户无须进行维护和担心硬盘驱动器的容量问题。Web 应用程序可自动进行软件和安全更新，这意味着它们始终是最新版本，并且面临的安全漏洞风险更低。

(4) 扩展性强。使用 Web 应用程序的企业可以在需要时添加用户，而无须配备额外的基础设施或昂贵的硬件。此外，绝大多数 Web 应用程序数据都存储在云中，这意味着企业无须投资

额外的存储容量来运行 Web 应用程序。

目前可通过两种通用方法构建 Web 应用程序：在服务器上执行大部分应用程序逻辑的传统 Web 应用程序，以及在 Web 浏览器中执行大部分用户界面逻辑的单页应用程序（Single Page Web Application, SPA），后者主要使用 Web API 与 Web 服务器通信。也可以将两种方法混合使用，最简单的方法是在更大型的传统 Web 应用程序中托管一个或多个丰富 SPA 类子应用程序。

16.1.2 Web应用特征

新式 Web 应用程序比以往承载着更高的用户期望和要求。当今的 Web 应用需要能在全球任何地方、在任意时刻可用，需要能在任何设备或屏幕尺寸上使用。Web 应用程序必须具有安全性、灵活性和可缩放性，以便满足高峰需求。如今复杂方案的处理日益需要丰富的用户体验，这建立在使用 JavaScript 并通过 Web API 进行有效通信的客户端的基础之上。新式 Web 应用程序具有以下特征。

1. 云托管和可扩展性

由于低内存、高吞吐量的特点，新式 Web 应用程序需要针对云（包括公共云、私有云以及任何云）进行优化。Web 应用程序占用空间更小，因此在相同硬件上可托管更多此类应用。并且，使用即用即付云托管服务时仅需为极少数资源付费。更高的高吞吐量意味着，通过相同硬件上的一个应用，Web 应用系统可服务更多客户，进而降低服务器和托管基础设施所需的投入。

2. 跨平台

大部分新式 Web 应用具有跨平台性，可在 Linux、macOS 以及 Windows 系统上运行。此功能为开发和部署的应用带来了新的选择。Docker 容器（包括 Linux 和 Windows）可托管 Web 应用程序，从而使应用程序可利用容器和微服务的优势。

3. 自动化测试

自动化测试能够利用自动化测试工具和框架，通过编写脚本来模拟用户在 Web 页面上的操作，自动化执行测试用例，比较预期结果和实际结果，包括一系列日志信息以及测试报告的自动生成，验证 Web 应用程序的正确性、稳定性和性能等指标。Web 应用自动化测试是一种高效、准确、可重复的测试方法，可以帮助开发人员和测试人员快速发现和解决 Web 应用程序中的缺陷和问题，同时也可以提高测试效率、降低测试成本、缩短测试周期。

4. 支持传统应用和 SPA

传统 Web 应用程序涉及极少的客户端行为，而是依赖服务器执行该应用可能需要执行的所有导航、查询和更新。用户执行的每个新操作会被转换为新的 Web 请求，其结果是最终用户浏览器中的完整页面重载。经典模型视图控制器（MVC）框架通常采用此方式，其中每个新请求对应一个不同的控制器操作，这些操作会反过来作用于模型并返回一个视图。特定页面上的一些单独操作可使用 AJAX（异步 JavaScript 和 XML）功能改进，但该应用的总体体系结构使用

了多个不同的 MVC 视图和 URL 终结点。此外, ASP.NET Core MVC 还支持 Razor Pages, 这是一种组织 MVC 样式页的较为简单的方法。

与之相反, 单页应用程序 (SPAs) 涉及极少动态生成的服务器端页面负载 (如有)。许多 SPA 在一个静态 HTML 文件中进行初始化, 此文件加载该应用启动和运行所需的 JavaScript 库。这些应用大量使用 Web API 处理数据需求, 并提供更为丰富的用户体验。BlazorWebAssembly 提供了一种使用 .NET 代码生成 SPA 的方法, 然后在客户端的浏览器中运行。

5. 简单的开发和部署

可使用简单的文本编辑器、命令行接口或者全功能开发环境 (如 Visual Studio) 编写 Web 应用程序。整体应用程序通常部署到单个终结点。可轻松将部署自动化, 作为持续集成 (CI) 和持续交付 (CD) 管道的一部分。除传统的 CI/CD 工具外, Microsoft Azure 还集成了 git 存储库支持, 并可在对特定 git 分支或标记作出更新时自动部署更新。Azure DevOps 提供了功能完备的 CI/CD 生成和部署管道, 而 GitHub Actions 为在那里托管的项目提供了另一个选择。

16.2 Web 应用架构设计

16.2.1 Web应用架构设计原则

Web 应用架构的设计原则可以帮助开发人员作出体系结构决策, 构建简洁、可维护的应用程序。一般而言, 在这些原则的指导下构建的应用程序各部分间可通过显式接口或消息传送系统进行通信, 它们并非松散耦合的离散组件。以下为 Web 应用架构的通用设计原则。

1. 分离关注点

分离关注点是开发时的指导原则。此原则主张根据软件执行的工作类型将软件分离。例如, 假设应用程序中包含两个逻辑, 其中一个逻辑标识要显示给用户的注意事项, 另一个以特定方式设置这些注意事项的格式, 使其更加显眼。负责选择为哪些事项设置格式的行为应与负责设置格式的行为区分开, 因为这两种行为只是碰巧彼此相关联的独立关注点。

从体系结构上来说, 按此原则有逻辑地构建应用程序应将核心业务行为与基础结构及用户界面逻辑区分开。理想情况下, 业务规则和逻辑应单独位于一个项目中, 且该项目不依赖应用程序中的其他项目。此区分操作有助于确保该业务模型易于测试, 且可在不与低级别实现详细信息紧密耦合的情况下逐步改进 (如果基础结构问题取决于业务层中定义的抽象时, 这也有帮助)。在应用程序体系结构的使用层背后, 关注点分离是核心设计思想。

2. 封装

应用程序的不同部分应通过封装与应用程序中的其他部分进行隔离。只要不违反外部协定, 应用程序的组件和分层应该能够在不中断其协作者的情况下调整其内部实现。正确使用封装有助于在应用程序设计中实现松散耦合及模块化, 因为只要维持相同的接口, 就可以用替代实现来替代对象和包。

在类中实现封装的方式是限制对该类内部状态的外部访问权限。如果外部参与者想操作对象的状态，则应通过明确定义的函数（或属性 **setter**）来进行操作，而非直接访问该对象的私有状态。同样，应用程序组件和应用程序本身应公开明确定义的接口供协作者使用，而非让协作者直接修改其状态。通过此方法，只要公共协定得到维护，就可以不断改进应用程序的内部设计，而无须担心会中断协作者。

可变全局状态与封装是对立的。不能依赖从一个函数中的可变全局状态获取的值在另一个函数中（甚至在同一个函数中）具有相同的值。理解可变全局状态问题是 C# 语言等编程语言支持不同范围规则的原因之一，从语句到方法再到类，这些规则被广泛使用。值得注意的是，对于依赖中央数据库在应用程序内部和应用程序之间进行集成的数据驱动的体系结构，它们本身选择依赖由数据库表示的可变全局状态。域驱动的设计和整洁的体系结构的一个重要考虑因素是如何封装对数据的访问，以及如何确保应用程序状态不会因直接访问其持久性格式而无效。

3. 依赖关系反转

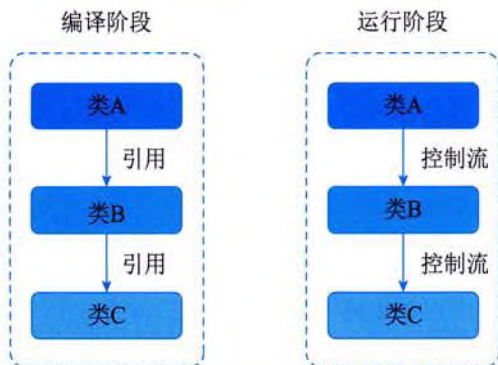


图 16-1 直接依赖关系图

应用程序中的依赖关系方向应该是抽象的方向，而不是实现详细信息的方向。大部分应用程序都是这样编写的：编译时依赖关系顺着运行时执行的方向流动，从而生成一个直接依赖项关系图。也就是说，如果类 A 调用类 B 的方法，类 B 调用 C 类的方法，则在编译时，类 A 将取决于类 B，而类 B 又取决于类 C，如图 16-1 所示。

应用依赖关系反转原则后，如图 16-2 所示，类 A 可以调用类 B 实现的抽象方法，让类 A 可以在运行时调用类 B，而类 B 又在编译时依赖于类 A 控制的接口（典型的编译时依赖项发生反转）。运行时，程序执行的流程保持不变，但接口引入意味着可以轻松插入这些接口的不同实现。

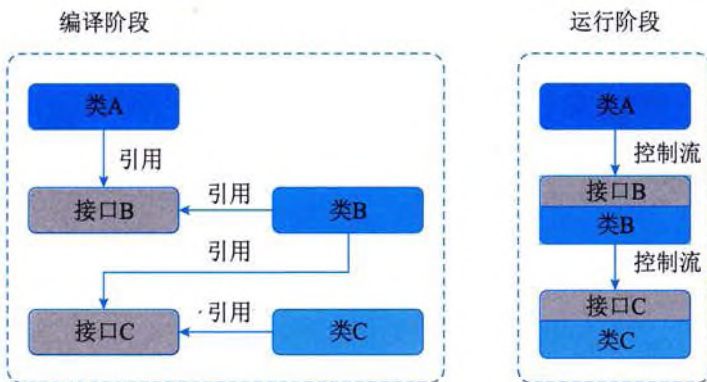


图 16-2 反转依赖关系图

依赖项反转是生成松散耦合应用程序的关键一环，因为可以将实现详细信息编写为依赖并

实现更高级别的抽象，而不是相反。因此，生成的应用程序的可测试性、模块化程度以及可维护性更高。遵循依赖关系反转原则可实现依赖关系注入。

4. 显式依赖关系

方法和类应显式要求正常工作所需的任何协作对象，一般将此称为显式依赖关系原则。通过类构造函数，类可以标识其实现有效状态和正常工作所需的内容。如果定义的类可供构造和调用，但仅在具备特定全局组件或基础结构组件时正常工作，则这些类对其客户端而言就不诚实。构造函数协定将告知客户端，它只需要指定的内容（如果类只使用无参数构造函数，则可能不需要任何内容），但随后在运行时，结果发现对象确实需要某些其他内容。

若遵循显式依赖关系原则，类和方法就会诚实地告知客户端其需要哪些内容才能工作。遵循此原则可以让代码更好地自我记录，并让代码协定更有利于用户，因为用户相信只要他们以方法或构造函数参数的形式提供所需的内容，他们使用的对象在运行时就能正常工作。

5. 单一责任

单一责任原则适用于面向对象的设计，但也可被视为类似于分离关注点的体系结构原则。它指出对象只应有一个责任，并且只能因为一个原因更改对象。具体而言，只在必须更新对象执行其唯一责任的方式时才更改对象。遵循这一原则有助于生成更松散耦合和模块化的系统，因为许多类型的新行为可以作为新类实现，而不是通过向现有类添加其他责任。添加新类始终比更改现有类安全，因为还没有任何代码依赖于新类。

在整体应用程序中，可以在高级别将单一责任原则应用于应用程序中的层。显示责任应位于 UI 项目中，而数据访问责任应位于基础结构项目中。业务逻辑应位于应用程序核心项目中，该项目易于测试，并且可以独立于其他责任进行逐步改进。

将此原则应用到应用程序体系结构及其逻辑终结点时，将获得微服务。给定的微服务应具有单一责任。一般而言，如果需要扩展系统的行为，最好通过添加其他微服务来实现，而不要向现有微服务添加责任。

6. 避免自我重复

应用程序应避免在多个位置指定与特定概念相关的行为，因为这种做法经常会导致出错。有时，如果要求发生变化，则要求更改此行为。至少有一个行为实例可能无法更新，系统行为可能不一致。

请将逻辑封装在编程构造中，而不要重复该逻辑。让此构造成为针对此行为的单一权限，并让应用程序中需要此行为的任何其他部分都使用新的构造。避免将恰巧重复的行为绑定在一起。例如，只因为两个不同的常数具有相同的值，如果从概念上讲，两个常数是指不同的内容，则并不意味着只应使用一个常数。复制总是比耦合到错误的抽象更可取。

7. 持久性无知

持久性无知是指需要保持不变的类型，但其代码不受所选择的持久性技术的影响。在 .NET 中的这种类型有时被称为普通旧 CLR 对象（Plain Old CLR Object, POCL），因为这种类型无须继承特定的基类或实现特定的接口。持久性无感知非常有用，因为它可以让相

同的业务模型以多种方式保持不变，让应用程序更加灵活。持久性选择可能会随着时间的推移而发生变化，从一种数据库技术变为另一种数据库技术，或除应用程序一开始具备的持久性形式之外还需要其他形式的持久性（例如，除相关数据库之外还需使用 Redis 缓存或 Azure Cosmos DB）。

违反此原则的一些示例如下：

- (1) 必需的基类。
- (2) 必需的接口实现。
- (3) 负责保存其自身的类（如活动记录模式）。
- (4) 所需的无参数构造函数。
- (5) 需要 `virtual` 关键字的属性。
- (6) 特定于持久性的必需特性。

要求类具有上述任何特性或行为，会增加要保持不变的类型和持久性技术的选择之间的耦合，从而增加将来采用新的数据访问策略的难度。

8. 有界上下文

有界上下文是领域驱动设计中的中心模式。它们可以将大型应用程序或组织分解为独立的概念模块，通过这种方式来解决复杂性问题。每个概念模块表示各自独立的上下文（因此有界），并且可以独立改进。理想情况下，每个有界上下文都能够为其中的概念自由选择它自己的名称，并对其自己的持久性存储具有独占访问权限。

至少，各 Web 应用程序应努力成为自己的有界上下文，为其业务模型提供自己的持久性存储，而不是与其他应用程序共享数据库。有界上下文之间的通信通过编程接口进行，而不是通过共享数据库进行，这样可以引发业务逻辑和事件来响应发生的更改。有界上下文会紧密映射到微服务，后者在理想情况下也作为其自己的单独有界上下文实现。

16.2.2 Web应用架构分类

Web 应用程序的体系结构可以根据软件开发和部署模式分为不同的类别。

1. 整体式架构

整体式架构是一种传统的软件开发模型（也称为 Web 开发架构），其中整个软件作为一段代码开发，运用传统的瀑布模型。这意味着所有组件都是相互依赖和互连的，并且每个组件都需要运行应用程序。若要更改或更新特定功能，需要重写并编译整个代码。

由于整体架构将整个代码视为单个程序，因此构建新项目、应用框架、脚本和模板并对其进行测试变得简单易行，并且部署也很容易。但是，随着代码量越来越大，管理或进行更新变得困难；即使是很小的更改，也需要经历 Web 开发架构的整个过程。由于每个元素都是相互依赖的，因此扩展应用程序并不容易。而且它并不可靠，因为单点故障可能会使应用程序瘫痪。当开发人员想要构建轻量级应用程序并且预算紧张时，整体式架构将达到这一目的。但是，当开发团队在同一地点工作而不是远程分布时，使用整体模型是有意义的。

2. 微服务架构

微服务架构解决了在整体环境中遇到的几个挑战。在微服务架构中，代码被开发为松散耦合的独立服务，通过 RESTful API 进行通信。每个微服务都包含自己的数据库并运行特定的业务逻辑，这意味着您可以轻松开发和部署独立的服务。

由于它是松散耦合的，微服务架构提供了更新 / 修改和扩展独立服务的灵活性。开发变得简单高效，并实现持续交付。开发人员可以快速适应创新。对于高度可扩展和复杂的应用程序，微服务是一个不错的选择。但是，使用运行时实例部署多个服务是一项挑战。当服务数量增加时，管理它们的复杂性也会增加。此外，微服务需应用共享分区数据库。这意味着应确保受事务影响的多个数据库之间的一致性。

3. 无服务器架构

无服务器架构是开发软件应用程序的模型。在此结构中，底层基础结构的预配由基础结构服务提供商管理。这意味着用户只需为正在使用的基础架构付费，而无须为空闲 CPU 时间或未使用的空间付费。无服务器计算降低了成本，因为资源仅在应用程序执行时被使用。扩展任务由云提供商处理。此外，后端代码得到简化。它减少了开发工作和成本，并加快了上市时间。多媒体处理、实时流媒体、聊天机器人 CI 管道、物联网传感器消息等，是无服务器计算的一些用例。在微服务架构中，可以使用 AWS Lambda、API Gateway 和 API Step Functions 执行无服务器计算。

在构建 Web 应用系统架构时，应根据系统需求与开发资源条件选择 Web 应用系统结构类型，图 16-3 详细展示了整体式架构和微服务架构的具体结构与区别。

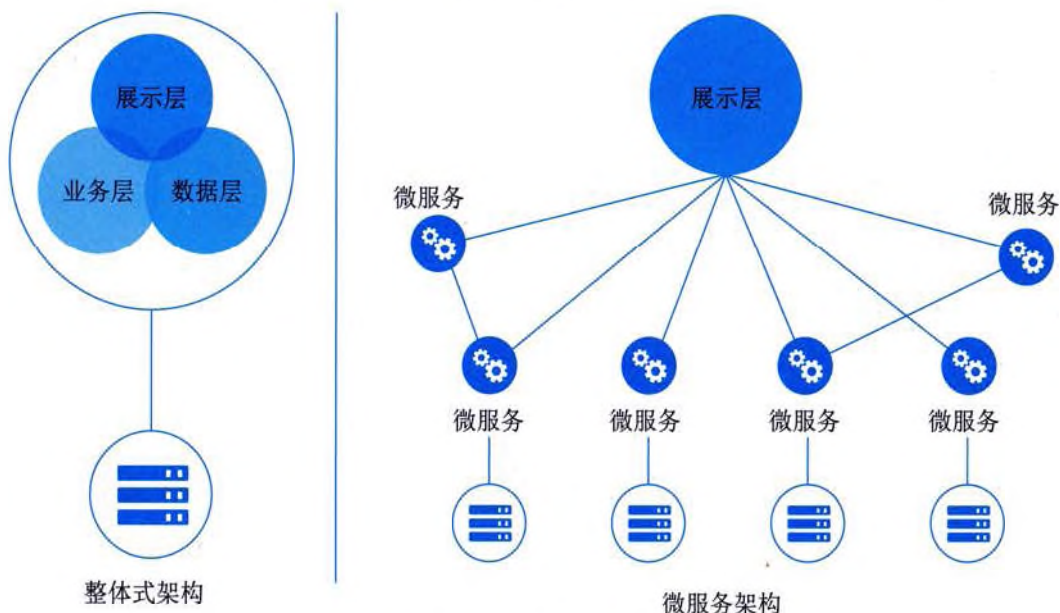


图 16-3 整体式架构和微服务架构

1) 何时使用整体式体系结构

整体式应用程序最适合要求非常简单的用例，该应用程序应处理有限的流量。这方面的一个例子是组织的内部税计算应用程序或类似的开放公共工具。在这些用例中，企业确信用户群和流量不会随着时间的推移呈指数级增长。在某些情况下，开发团队决定从整体式体系结构开始，然后横向扩展到分布式微服务体系结构。

2) 何时使用微服务架构

微服务架构最适合复杂的用例，以及预计未来流量呈指数级增长的应用程序，如功能复杂的社交网络应用程序。典型的社交网络应用程序具有各种组件，如消息传递、实时聊天、实时视频流、图像上传、喜欢、共享功能等。在这种情况下，分别开发每个组件的同时需要牢记单一责任和关注点分离原则。选择整体式架构或微服务架构在很大程度上取决于实际的应用例。

16.2.3 Web应用架构模式

1. B/S 架构

B/S 架构的全称为 Browser/Server，即浏览器 / 服务器结构。Browser 指的是 Web 浏览器，极少数事务逻辑在前端实现，主要事务逻辑在服务器端实现。B/S 架构的系统无须特别安装，只需要有 Web 浏览器即可。该体系结构基于请求 - 响应模型。客户端将请求发送到服务器以获取信息，服务器对此进行响应。

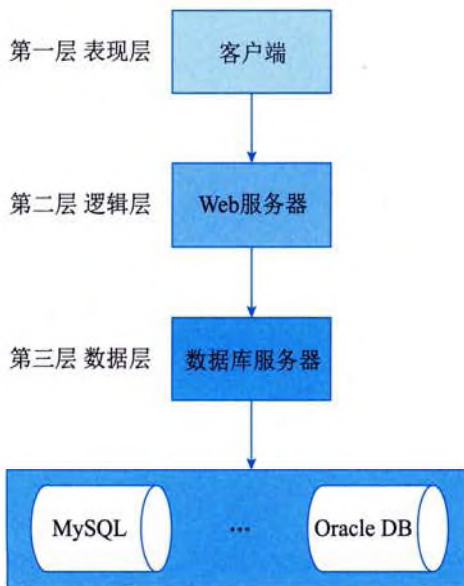


图 16-4 B/S 架构图

B/S 架构一般分为三层：第一层是表现层，主要完成用户和后台的交互及最终查询结果的输出功能；第二层是逻辑层，主要是利用服务器完成客户端的应用逻辑功能；第三层是数据层，主要是接受客户端请求后独立进行各种运算。B/S 的架构图如图 16-4 所示。

2. P2P 架构

P2P 架构是一种网络，是两个或多个客户端不经过服务器而直接通信的架构。可能首先会使用服务器，以让客户端间查找到对方，但在此之后就不再需要服务器了。具体分两种不同的形式：一种是完整连接拓扑架构，指的是每个客户端与其他客户端之间都必须有连接，信息可以直接在用户间交换；另一种是环形拓扑架构，指的是信息只有通过一个或多个客户端才能传递的架构。P2P 架构中的所有计

算机都具有相同的权限。因此，即使某些计算机 / 节点出现故障，网络和通信仍然处于正常运行状态。P2P 的基本架构如图 16-5 所示。

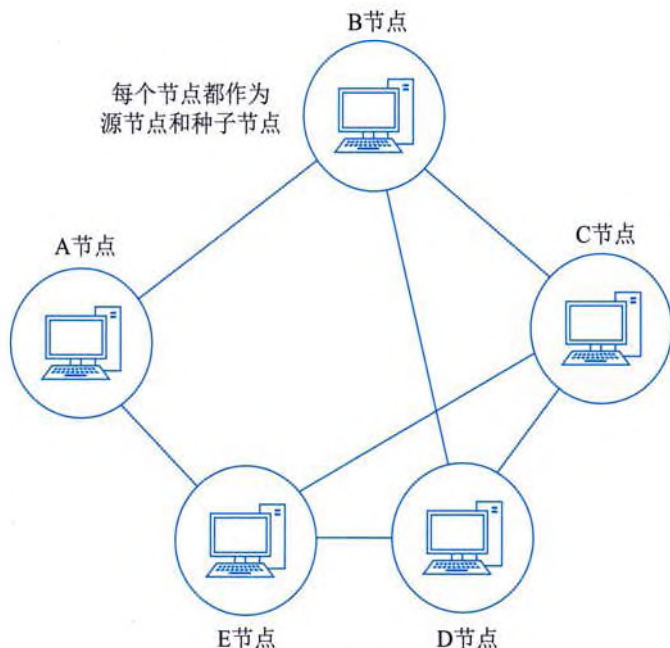


图 16-5 P2P 架构图

3. MVC/MVP 架构

MVC (Model-View-Controller) 体系结构是一种软件体系结构模式，其中应用程序逻辑根据功能分为三个组件。这些组件称为：

- 模型 (model)，表示数据在数据库中的存储方式；
- 视图 (view)，用户可见的组件，例如输出或 GUI；
- 控制器 (controller)，充当模型和视图之间接口的组件。

MVC 体系结构不仅用于桌面应用程序，还用于移动和 Web 应用程序。MVP (Model-View-Presenter) 模式是由 MVC 模式派生出的一种设计模式，其中 Model 提供数据，View 负责显示，Controller/Presenter 负责逻辑的处理。MVP 从 MVC 演变而来，通过表示器将视图与模型巧妙地分开。在该模式中，视图通常由表示器初始化，它呈现用户界面 (UI) 并接收用户所发出的命令，但不对用户的输入做任何逻辑处理，而是仅将用户输入转发给表示器。通常每个视图对应一个表示器，但是也可能一个拥有较复杂业务逻辑的视图会对应多个表示器，每个表示器完成该视图的一部分业务处理工作，降低了单个表示器的复杂程度，一个表示器也能被多个有着相同业务需求的视图复用，增加单个表示器的复用度。表示器包含大多数表示逻辑，用以处理视图，以及与模型交互以获取或更新数据等。模型描述了系统的处理逻辑，模型对于表示器和视图一无所知。在 MVP 里，Presenter 完全把 Model 和 View 进行了分离，主要的程序逻辑在 Presenter 里实现。Presenter 与具体的 View 是没有直接关联的，而是通过定义好的接口进行交互，从而使得在变更 View 时可以保持 Presenter 不变。

4. 微服务架构

微服务是由以单一应用程序构成的小服务，自己拥有自己的进程与轻量化处理，服务依据业务功能设计，以全自动的方式部署，与其他服务使用 HTTP API 通信。同时服务会使用最小规模的集中管理（如 Docker）能力，服务可以用不同的编程语言与数据库等组件实现。微服务架构区别于传统的单体软件架构，是一种为了适应当前互联网后台服务的三高需求：高并发、高性能、高可用而产生的软件架构。与微服务相对的另一个概念是传统的单体式应用程序（Monolithic Application），其内部包含了所有需要的服务。而且各个服务功能模块有很强的耦合性，彼此相互依赖，很难拆分和扩容。在微服务架构中，不同的功能 / 任务被拆分为单独的模块 / 代码库，这些模块 / 代码库相互协同工作，形成一个整体的大型服务。与单体架构相比，这种特殊的架构有助于更轻松、更干净的应用程序维护、功能开发、测试和部署。微服务的架构如图 16-6 所示。

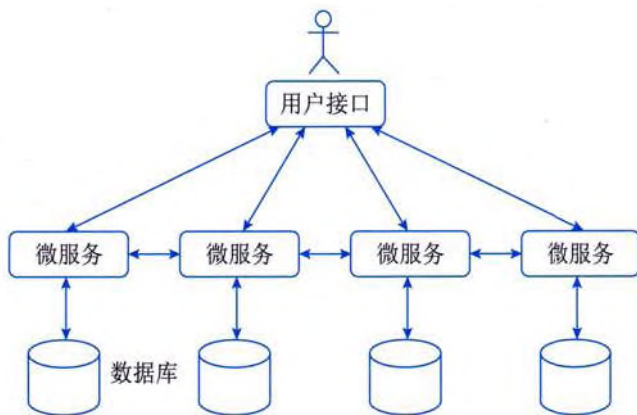


图 16-6 微服务结构图

微服务架构相对单体服务有很多优点，如技术异构性、隔离性、可扩展性、部署简单等。但这种架构也存在一些缺点。微服务把单体应用拆分成很多小的分布式服务，不可避免地会存在服务管理问题。为了克服微服务架构存在的缺点，一些技术概念相继被提出。

1) 服务注册与发现

微服务之间相互调用完成整体业务功能，如何在众多微服务中找到正确的目标服务地址，这就是所谓服务发现功能。常用的做法是服务提供方启动时把本服务的地址上报给服务注册中心，这就是服务注册。服务调用方订阅服务变更通知，动态接收服务注册中心推送的服务地址列表。

2) 服务监控

大型微服务架构的服务运维是一大挑战。服务运维人员需要实时掌握服务运行中的各种状态，一般需要控制面板监测服务的内存使用率、调用次数、健康状况等信息。这就需要系统有一套完备的服务监控体系，包括拓扑关系、监控（Metrics）、日志监控（Logging）、调用追踪（Trace）、告警通知、健康检查等。

3) 服务容错

任何服务都不能保证 100% 不出问题，生产环境复杂多变，服务运行过程中不可避免地会

发生各种故障（宕机、过载等），工程师能够做的是在故障发生时尽可能降低影响范围、尽快恢复正常服务，因此需要引入熔断、隔离、限流和降级、超时机制等服务容错机制来保证服务持续可用性。

16.3 Web 应用开发框架

16.3.1 开发框架简介

框架（Framework）是一组组件的综合，这些组件相互协作，为一类相关应用提供了可重用的框架结构，描述了另一种重用架构知识的方式，支持细节设计和代码重用，如 MMC、MS Script Engine、Spring、Struts 和 Hibernate 等。框架可以进行实例化，部分完成系统或者子系统，并提供了构建系统的基本构建块，还为实现特定功能定义了适应点。在面向对象系统中，框架通常由抽象类和具体类组成。

使用 Web 应用框架的 Web 应用程序开发人员可以在一个通用功能已经实现的框架基础上开发具体的应用程序，使开发者可以集中精力开发系统所需的商业逻辑，从而降低系统开发的难度，提高 Web 应用开发的效率，使系统的可扩展性、可维护性和灵活性得到提高。例如，在新闻系统中，采用了 Struts、Spring 和 Hibernate 三个开发框架，实现了一个典型的 MVC 架构。

16.3.2 Java EE 开发框架

基于 Java EE 的 Web 应用开发的框架很多，常用的有 Spring、Struts、Hibernate、WebWork 等，本节简要介绍如下几种框架。

1. SSH

SSH 是 Spring、Struts 和 Hibernate 的结合，是目前基于 Java EE 的企业级 Web 应用开源框架，其轻量级架构如图 16-7 所示。

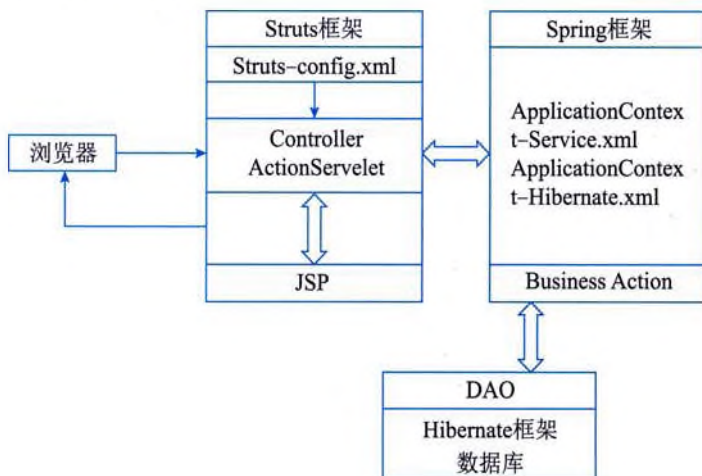


图 16-7 基于 Struts、Spring 和 Hibernate 的轻量级架构

1) Spring

Spring 框架是为了解决企业应用开发的复杂性而创建的开源框架，其主要优势之一就是分层架构。分层架构允许使用者选择使用哪个组件，同时为 J2EE 应用程序开发提供集成的框架。Spring 框架易于使用并整合各类框架，能统一配置和部署、灵活并且可扩展、测试简单、开发成本低。Spring 框架依赖注入机制，可以在运行期间为组件配置所需资源，而无须在编写组件代码时就加以指定，从而在相当程度上降低了组件之间的耦合性。客户代码仅面向接口编程，而无须知道实现类的具体名称。同时，开发人员可以很简单地通过修改配置文件来切换具体的底层实现类，自定义组件并不需要实现框架指定的接口，易于将组件从 Spring 框架中脱离，甚至不需要做任何修改，方便转移到其他框架中去，使得组件间的依赖关系减少，极大改善了代码的可重用性。

2) Struts

Struts 是 Apache 软件组织负责开发的一个基于 MVC 模式的企业级 Web 应用架构的开源框架，主要采用 Java Servlet、JSP 和 XML 等技术来实现。Struts 把 Servlet、JSP、自定义标签和信息资源（Message Resources）整合到一个统一的框架中，使利用者在开发时减少自己的编码量，节约开发时间。Struts 中使用 Plugin 实现 Web 应用设计架构中的接口层，ActionServlet、Action 和 ActionForm 等类实现抽象层，后台实现各种实现层的功能。Struts 框架中的模型、视图和控制器三部分通过 struts-config.xml 配置文件联系在一起。采用 Struts 框架可以加快开发速度、增强系统灵活性并降低系统的耦合性；使用标记能够把逻辑处理代码从页面中分离出来，简化页面开发；提高代码的可重用性。

3) Hibernate

Hibernate 是开源对象关系映射框架，对 JDBC 进行了非常轻量级的对象封装，使 Java 程序员可以使用对象编程思维来操作数据库。Hibernate 查询语言（HQL）被设计成 SQL 的一个轻量级面向对象扩展，是对象和关系之间的桥梁。Hibernate 也支持用原始 SQL 或基于 Java 的标准和示例查询表达查询。Hibernate 使用 XML（.hbm.xml）文件把 Java 类映射到表，把 Java Bean 属性映射到数据库表。通过 JDBC 技术，支持所有的 SQL 数据库管理系统。

2. JSF

JSF（Java Server Faces）是用于构建 Web 应用程序的标准 Java 框架，通过提供标准可扩展的用户界面组件、易配置的页面导航、方便的数据验证和转换、自动化 Bean 管理、事件处理、方便的错误处理以及内置对国际化的支持来加速 Web 应用开发。JSF 中，Model 是进行业务操作的部分，用来实现业务逻辑，一般使用 JavaBean 或 EJB 来建立复杂的企业应用；View 是由 JSF 标签构成的 JSP 页面组成，通过一个字符串松耦合 Controller，通过发送事件来间接调用 Controller 的逻辑；Controller 主要包括 FacesServlet、配置文件和 action 处理器，其中 FacesServlet 负责从客户端接收请求，执行一组合理步骤（重建组件树、应用请求值、处理验证、更新模型值、调用应用、呈现响应）来准备和派发响应。

16.3.3 .NET开发框架

.NET 框架是由微软开发的一个致力于敏捷软件开发、快速应用开发、平台无关性和网络透

明化的软件开发平台，是一个可以构建、发布以及运行 Web 服务及其他应用程序的环境，它提供了托管执行环境、简化的开发和部署以及与各种编程语言的集成。.NET 框架的两个关键组件为 CLR 和 .NET 框架类库，其中，.NET 框架类库包括 ADO.NET、ASP.NET、Windows 窗体和 Windows Presentation Foundation (WPF)。

16.3.4 Web 层开发框架

1. WebPage 3.0

WebPage 3.0 是基于组件的、可视化的、轻量级的 Web 开发框架，基于标准技术，有极好的稳定性和可扩展性。WebPage 3.0 基于 MVC 模式，重点关注 View 部分，达到可视化开发和最大限度的重用，主要使用 Java、JSP、Servlet、HTML、JavaScript 和 XML 等技术。WebPage 3.0 开放组件设计接口，可以自由开发能在 WebPage 3.0 中使用的组件，而且开发组件非常简单，WebPage 3.0 能大大提高 Web 层的开发速度。

2. AJAX 框架

AJAX 作为重要的 Web 2.0 技术被广泛采用。随之也出现了很多 AJAX 框架，如 Prototype、jQuery、Mootools、DOJO、Ext JS 以及 Ajax.NET 和 AFAX 等。

1) Prototype

Prototype 是一个由 Sam Stephenson 创建的 JavaScript 类库，它定义了 JavaScript 的面向对象扩展、DOM 操作 API 和事件等。Prototype 的特点是功能实用而且规模较小，非常适合在中小型的 Web 应用中使用。开发 AJAX 应用需要编写大量的客户端 JavaScript 脚本，而 Prototype 框架可以大大简化 JavaScript 代码的编写工作。同时，Prototype 具备兼容各个浏览器的特性，使用该框架无须考虑浏览器兼容性的问题。

2) jQuery

jQuery 是由美国人 John Resig 在 2006 年创建的一种轻量级的免费 JavaScript 框架，它为 Web 的 JavaScript 开发提供辅助功能。jQuery 语法设计简单，使用方便，用户可以更加方便地处理 HTML、DOM、Events，实现动画效果，并且它提供对 AJAX 的支持。jQuery 还兼容各种浏览器，同时文档说明详细，还有许多成熟的插件可供选择，逐渐成为目前深受开发人员欢迎的 AJAX 开发框架之一。

3) Mootools

Mootools 是一款结构紧密的模块化 JavaScript 框架，最大特点是视觉效果和变换特性。MooTools 是一个简洁、模块化、面向对象的开源 JavaScript 框架，它能够帮助用户更快、更简单地编写可扩展和兼容性强的 JavaScript 代码。MooTools 跟 Prototype 类似，语法几乎完全一样，而且功能更强大，可以实现优雅且可定制的动画，同时具有完整的 API 文档以及丰富的范例。

4) DOJO

DOJO 是由 JavaScript 编写的开源工具包，它的组件丰富，是一个强大的面向对象 JavaScript 框架，其目的是简化动态 Web 应用开发。开发人员使用其底层 API 和兼容层编写兼

容的 JavaScript 和简化复杂脚本。DOJO 的事件系统、I/O API 和通用语言增强形成了强大的编程环境。DOJO 的包加载机制和模块化的库结构，能保持更好的扩展性，提高执行性能，减轻了用户开发的工作量，并保持一定的灵活性（用户可以自己编写扩展）。DOJO 目前支持 IE、Firefox、Safari、Opera、Chrome、iOS 和 Android 浏览器，已经是众多开源框架的选择，包括 WebWork、Tapestry、Eclipse ATF、MyFaces。

5) Ext JS

Ext JS 是由 JackSlocum 开发的一种主要用于创建前端用户界面、与后台技术无关的前端开源 AJAX 框架，允许用户使用 AJAX、DHTML 和 DOM 脚本构建富交互的 Web 应用，且可用于 .NET、Java 和 PHP 等各种开发语言开发的应用中。Ext JS 是一个重量级的框架，它包含大量的 UI 元素。Ext JS 的 UI 组件模型和开发理念成型于 Yahoo 组件库 YUI 和 Java 平台上的 Swing，为开发者屏蔽了大量跨浏览器方面的处理。相对来说，Ext JS 开发者直接针对 DOM、W3C 对象模型开发 UI 组件轻松。

6) AFLAX

AFLAX 是基于 AJAX 的“派生/合成”式（derivative/composite）技术，是一种用于 Macromedia 公司 FlashTM 平台的 JavaScript 库，使开发人员能够将 JavaScript 和 Flash 进行结合，创建出 AJAX 类型应用。

16.4 Web 应用系统开发

Web 应用开发指的是通过 Web 技术构建 Web 应用程序的过程。构建阶段包括一系列的选择、编码、内容创建、集成、重构以及测试（考虑到测试的重要性，故单独在第 16.6 节对其进行介绍）活动，以构建出可以部署并交付给最终用户使用的 Web 应用。选择什么样的实现技术，是 Web 应用成功的一个关键因素，最基本的原则是从内容、表示、功能分开考虑，而且需要考虑架构中分布式以及与其他系统之间的交互。因此，需要了解各种技术的特点，这些技术适用什么样的架构，以及细粒度增量的部署。

16.4.1 Web 应用系统通信协议

Web 上计算机之间通过特定的通信协议完成通信。通信协议是网络上所有设备（如 Web 服务器、计算机及交换机、路由器、防火墙等）之间通信规则的集合，规定了通信时信息必须采用的格式及其含义。Web 应用通信协议是 Web 应用开发的基础，常用的 Web 应用通信协议有 HTTP、MIME、RTP/RSTP 和 MMS 等。

1. HTTP

HTTP（HyperText Transfer Protocol，超文本传输协议）是基本的 Web 通信协议，属于应用层的面向对象的协议，是一个基于文本的无状态协议，是运行在 TCP/IP 上的网络应用层协议，是客户端浏览器与 Web 服务器之间的应用层通信协议。其基本功能是完成 Web 应用中必要的文件传输和基于 Web 的动态交互应用。

HTTP 无连接、无状态，客户和服务端之间的交互不保留客户状态信息。对于交互型 Web 应用而言，需要借助一些其他技术来区分并发用户以及同一用户的相关联请求。一般情况下，通过 session（会话）来定义特定用户和服务端之间一定时间范围的相关 HTTP 请求。对 session 进行跟踪实现用户状态通常采用 URL 重写和 cookies 两种方法。

2. RTP/RTSP

RTP（Real-time Transport Protocol，实时传送协议）是一个网络传输协议，它是由 IETF 的多媒体传输工作小组 1996 年在 RFC 1889 中公布的。其详细说明了在 Internet 上传递音频和视频的标准数据包格式。RTP 一开始被设计为一个多播协议，但后来被用在很多单播应用中。RTP 常用于流媒体系统（配合 RTCP）、视频会议和一键通（Push to Talk）系统（配合 H.323 或 SIP），使其成为 IP 电话产业的技术基础。RTP 是建立在用户数据报协议上的，和 RTP 控制协议（RTCP）一起使用。

RTSP（Real Time Streaming Protocol，实时流传输协议）是 TCP/IP 体系中的一个应用层协议，定义了一对多应用程序如何有效地通过 IP 网络传送多媒体数据，用来控制音频或视频的实时发送，并允许同时控制多个流。RTSP 在体系结构上位于 RTP 和 RTCP 之上，使用 TCP 或 RTP 完成数据传输。

3. SMTP 和 POP3

SMTP（Simple Mail Transfer Protocol，简单邮件传输协议）是一种提供可靠且有效的电子邮件传输的协议，是建立在 FTP 文件传输服务上的一种邮件服务，主要用于传输系统之间的邮件信息并提供与来信有关的通知。SMTP 设计基于以下通信模型：针对用户的邮件请求，发送 SMTP 与接收 SMTP 之间建立一个双向传送通道，接收 SMTP 可以是最终接收者也可以是中间传送者，即“SMTP 邮件中继”。SMTP 是基于 TCP 服务的应用层协议，用户直接使用的使用于编写和发送的客户端软件，而 SMTP 服务器运行在远程的 Web 应用上。

POP3（Post Office Protocol V3，邮局协议版本 3）用于电子邮件的接收，它使用 TCP 的 110 端口。该协议规定了与 POP3 服务器进行对话的一系列命令和过程标准，它是因特网电子邮件的第一个离线协议标准，POP3 允许用户从服务器上把邮件存储到本地主机上，同时根据客户端的操作删除或保存在邮件服务器上的邮件。POP3 服务器接收的是用户发送至 SMTP 服务器的邮件。

以上协议只是 Web 上一些常用的通信协议，除了这些协议以外，还有 RIP、NFS、DNS 协议等。

16.4.2 Web应用系统数据存储

1. 缓存系统

缓存系统是一种本地数据存储，有助于快速访问应用程序服务器的数据，而不是每次都联系数据库。在传统设置中，数据存储于数据库中。当用户发出请求时，应用服务器会从数据库中请求该数据并将其呈现给用户。当再次请求相同的数据时，服务器应再次执行重复且耗时的相同过程。通过将此信息存储在临时缓存中，应用可以快速向用户呈现数据。

缓存系统可以设计为如下 4 种模型：

- (1) 应用程序服务器缓存。应用程序服务器旁边的内存中缓存（对于具有单个节点的应用程序）。
- (2) 全局缓存。所有节点访问单个缓存空间。
- (3) 分布式缓存。缓存分布在节点上，其中一致哈希函数用于将请求路由到所需的数据。
- (4) 内容交付网络（CDN）。用于交付大量静态数据。

2. 云存储

云存储通过集群应用、网格技术或分布式文件系统等功能，网络中大量各种不同类型的存储设备通过应用软件集合起来协同工作，共同对外提供数据存储和业务访问功能的一个系统，保证数据的安全性，并节约存储空间。简单来说，云存储就是将存储资源放到云上供使用者存取的一种新兴方案。使用者可以在任何时间、任何地方，通过任何可连网的装置连接到云上方便地存取数据。

Amazon S3 是全球 Web 应用程序架构 AWS 环境中常用的云存储解决方案。Amazon Simple Storage Service (S3) 是一种云存储服务，具有灵活性、经济高效、持久、安全的特点，并提供高可用性和高可扩展性。在 AWS 中，云存储单元称为“存储桶”，其中存储对象。创建桶时，将部署在用户指定的地域。完成部署并将对象添加到存储桶后，用户可以选择存储类型以及版本控制、生命周期策略、存储桶策略等功能。AWS 负责一组对象的生命周期管理，包括 IAM 策略和数据保护。

Azure Cloud Storage 是 Microsoft Azure 提供的另一种流行的云存储服务。Azure 存储最有优势的一点是具有 99.95% 正常运行时间的高可用性和高安全性。价格为 0.18 美元每吉字节每月，性价比很高。Azure 附带了全面的管理访问和开发人员工具堆栈，可帮助组织无缝协调整个业务运营。

谷歌云存储是谷歌提供的云存储产品，价格为 0.02 美元每吉字节每月。它在多个地区可用，提供高耐用性，并可轻松与其他 Google 服务集成。

3. CDN (CloudFront)

内容交付网络（CDN）是安装在各个地理位置的服务器网络，用于更快、更好地向用户交付内容。用户的请求不是联系中央服务器，而是路由到存储内容的缓存版本的 CDN 服务器。因此，站点速度和性能得到提高，数据包丢失减少，服务器负载减少。它还支持受众细分和高级 Web 安全性。

CloudFront 是一种流行的 CDN 服务，适用于 Web 应用程序架构 AWS。它充当分布式缓存，以提供更高的速度、低延迟和更好的客户体验。考虑到 AWS 的全球存在，CloudFront 为用户提供了更广泛的地域覆盖。CloudFront 与其他 AWS 服务（如 Amazon EC2、AWS Lambda、Amazon CloudWatch、Amazon S3 等）可以很好地集成，使开发人员的工作更轻松。它灵活、易于设置并提供高可扩展性，还具有弹性服务和分析功能，开发人员也可以控制对内容的访问。

Azure CDN 是来自 Microsoft Azure 云平台的流行内容交付网络，易于配置和使用，并提供低延迟。

谷歌的内容交付网络称为 Cloud CDN。它利用全球分布的边缘服务器在使用位置缓存内容,以高速交付内容。

CloudFlare 是另一种流行的 CDN 服务。尽管 CloudFlare 主要提供强大的 DNS 服务,并且不是传统的内容交付网络,但它充当反向代理,通过其全球数据中心网络路由流量。

4. 负载均衡器

负载均衡器是一种服务,它根据可用性或预定义策略将流量负载分布在不同的服务器之间来平衡流量负载。当负载均衡器中收到用户请求时,它会根据可用性和可伸缩性检索服务器的运行状况,并将请求路由到最佳服务器。负载均衡器可以是硬件组件,也可以是软件程序。

负载均衡可以通过如下两种方式完成:

- (1) TCP/IP 级别负载均衡,基于 DNS 的负载均衡。
- (2) 应用级负载均衡,基于应用负载的负载均衡。

Web 系统设计中的负载均衡可以分为软件负载均衡和硬件负载均衡。软件负载均衡解决方案是指在一台或多台服务器相应的操作系统上安装一个或多个附加软件来实现负载均衡,它的优点是基于特定环境,配置简单,使用灵活,成本低廉,可以满足一般的负载均衡需求。软件负载均衡又分为四层负载均衡和七层负载均衡。四层负载均衡中性能较为突出的有 LVS 和 Haproxy 等, LVS 是在 Linux 内核层进行数据交换,高并发能力接近硬件负载均衡的水平。但四层交换也有其不足之处,如不能检测后端服务器存活情况,不支持正则动静分离。七层负载均衡中性能较优的有 Haproxy 和 Nginx 等,两者都支持虚拟主机,在负载均衡的同时都有保持 session 的方案,并且可以进行动静分离。但显而易见,其性能不如四层交换。因此,根据不同网站的需求在负载均衡上的部署方案也不同。硬件负载均衡解决方案是直接在服务器和外部网络间安装负载均衡设备,通常称之为负载均衡器,由于专门的设备完成专门的任务,独立于操作系统,整体性能得到极大提高,加上多样化的负载均衡策略,智能化的流量管理,可达到最佳的负载均衡需求。负载均衡器有多种多样的形式,除了作为独立意义上的负载均衡器外,有些负载均衡器集成在交换设备中,置于服务器与 Internet 链接之间,有些则以两块网络适配器将这一功能集成到 PC 中,一块连接到 Internet 上,一块连接到后端服务器群的内部网络上。一般而言,硬件负载均衡在功能、性能上优于软件方式,不过成本昂贵。常见的有 NetScaler、F5、Radware、Array 等商用的负载均衡器。

5. 消息队列

消息队列是一个缓冲区,它异步存储消息,并促进 Web 应用程序中不同服务之间的通信。在当今的微服务环境中,软件是在更小、模块化和独立的构建块中开发的,这些构建块使用 RESTful API 相互通信。这些块之间的通信是使用消息队列协调的。软件组件连接到这些消息队列的末端,以发送和接收消息并处理它们。消息队列提供精细的可伸缩性,简化解耦过程并提高可靠性和性能。

事件系统派遣机制设计可分为无独立派遣模块和有独立派遣模块两大类设计方式。基于消息队列的点对点模式属于有派遣模块设计方式类别中的一种,在该方式中,消息生产者生产消息并发送到消息队列 (Queue) 中,然后消息消费者从 Queue 中取出并且消费消息。消息被消

费以后，Queue 中不再有存储，所以消息消费者不可能消费到已经被消费的消息。Queue 支持存在多个消费者，但是对一个消息而言，只会有一个消费者可以消费。例如，外卖系统采用的就是基于消息队列点对点模式的事件派遣机制。

16.4.3 Web应用系统客户端技术

Web 客户端的主要任务是通过浏览器来展现页面内容并实现与用户的交互，HTML 是信息展现的有效载体。为了提高客户端的功能性，出现了脚本、DOM 等概念；为了提高开发人员对信息展现格式的控制能力，出现了 CSS 等技术。常用的客户端开发技术主要有 HTML、CSS 等基本开发技术，客户端脚本、DHTML、DOM 等主要开发技术和 ActiveX、Applet 等扩充开发技术，其开发的代码在客户端（主要是浏览器）中执行。Web 客户端开发的主要任务是设计与开发 Web 应用的 Web 页面，通过设计格式、布局以实现数据的展现和用户交互。以下将简单分析 HTML、CSS 和 DOM 等一些客户端常用开发技术。

1. HTML/HTML5

HTML (HyperText Markup Language, 超文本标记语言) 是一种通用的标记语言，标记用符号“<”和“>”括起来。它简单易懂、易于实现，允许 Web 页面设计人员建立文本与图片相结合的复杂页面，这些 Web 页面可以被网上任何人浏览，而不管使用的是什么类型的计算机或浏览器。HTML 定义了很多标记来表示不同的语义，如 <H1> 表示一级标题。

HTML 也有以下几个主要缺点：

(1) 表现过于简单。HTML 文件将数据和数据的展示集中在一起，形式较为简单，尽管具有表达脚本、表格等功能，但很难表达复杂的形式。

(2) 链路容易中断。链宿地址改变后，链源不能自动纠正。

(3) 检索时所花的时间较长。检索到的内容针对性较差，返回的结果较多。

(4) 扩展性差。HTML 的标记集合是固定的，不允许用户自行定义他们自己的标识。由于网络技术发展得非常快，不断有新的数据格式的文档产生，这就要求要有一种比较灵活的标签机制才能满足网络信息不断发展的要求，但 HTML 不允许用户根据需要来创建新的标记，更无法表示许多特殊行业的数据。

(5) 缺乏语义性。HTML 是一种标记技术，不能很好地揭示信息内容的本质，计算机无法知道各段文本的确切含义。HTML 在设计上是用来展示内容和手工浏览 Web 页面的，不适合用作网络信息资源的自动化组织管理。

这些缺点通过一系列扩展来进行解决，如采用 CSS、XHTML、XML 等。

随着 Web 标准和技术的发展，出现了 HTML 的新一代版本 HTML5。HTML5 不是为了内容展示，而是为了支撑广泛的 Web 应用，因此，它支持新的元素、结构和语义。HTML5 是 Web 开发标准最巨大的飞跃，它并非仅用来表示 Web 内容，它的新使命是将 Web 带入一个成熟的应用平台，在 HTML5 平台上，视频、音频、图像、动画，以及与计算机的交互都被标准化。HTML5 功能强大，让内容创作者只要使用基于标准的 HTML5，就能使页面使用可扩展图形、动画和多媒体。

2. XML

XML(eXtensible Markup Language, 可扩展标记语言)是 W3C 制定的一种简单、跨平台的、依赖于内容的技术,是目前处理结构化文档信息的有力工具。XML 文档可以是服务器端的文件,也可以是如 Web 服务环境中的计算机系统之间传输的数据。XML 的可扩展性强,它是一种元语言,可用于创建或定义其他个性化的标记语言,如 RSS、MathML,甚至 XSLT。XML 的标记可以自定义,以提供更多的数据操作。

Web 本身就是一个最大的分布式应用系统,因而,XML 技术在 Web 上的优势明显。Web 应用借助 XML 格式交换信息,能很好地解决分布式架构上的信息交换。XML 在 Web 应用服务器端应用很广。

例如,信息标准化是解决“信息孤岛”的重要途径,也是不同的管理信息系统之间数据交换和互操作的基础。作为信息化标准的一项关键技术,在目前流行的可扩展标记语言(XML)、超文本标记语言(HTML)、通用标记语言(GML)、Web 服务描述语言(WSDL)中,XML 以开放的自我描述方式定义了数据结构,在描述数据内容的同时能突出对结构的描述,从而体现出数据之间的关系。这样组织的数据对于应用程序和用户都是友好的、可操作的。

3. DHTML

DHTML(Dynamic HTML, 动态 HTML)是近年来 Web 发展进程中最具有实用性的创新之一,它是无标准的,是一种通过各种技术的综合发展而得以实现的概念,这些技术包括 JavaScript、VBScript、DOM、Layers 和 CSS 等。

DHTML 就是当 Web 页面从 Web 服务器下载后无须再经过服务器的处理,而在浏览器中直接动态更新 Web 页面的内容、排版样式、动画等。例如,当鼠标指针移至文章段落中时,段落能够变成蓝色,或者当用户点击一个超链后会自动生成一个下拉式的子超链目录。

基于 Web 2.0 的 Web 应用十分强调用户的体验与感受,AJAX 是目前广泛使用的表现层技术。借助于 AJAX,可以在用户单击按钮时,使用 JavaScript 和 DHTML 立即更新用户界面,并向服务器发出异步请求,以执行更新或查询数据库。当请求返回时,就可以使用 JavaScript 和 CSS 来相应地更新用户界面,而不是刷新整个页面。最重要的是,用户甚至不知道浏览器正在与服务器通信,Web 站点看起来是即时响应的。使用 AJAX 的最大优点就是能在不更新整个页面的前提下维护数据,这使得 Web 系统更为迅捷地回应用户动作,并避免了在网络上发送那些没有改变过的信息。

进行 AJAX 开发时,需要慎重考虑网络延迟。不给予用户明确的回应,没有恰当的预读数据,或者对 XMLHttpRequest 的不恰当处理,都会使用户感到延迟,这是用户不希望看到的,也是他们无法理解的。通常的解决方案是,使用一个可视化的组件来告诉用户,系统正在进行后台操作并且正在读取数据和内容。

4. CSS

CSS(Cascading Style Sheets, 层叠样式表)是由 W3C 定义和维护的标准,是一种用来为结构化文档(如 HTML 文档或 XML 应用)添加样式(字体、间距和颜色等)的计算机语言。

一个 Web 页面的读者和作者都可以使用 CSS 来决定文件的颜色、字体、排版等显示特性。

CSS 最主要的目的是将文件的结构（用 HTML 或其他相关的语言写的）与文件的显示（CSS）分隔开来。CSS 还可以使用其他的显示方式，如声音（假如浏览器有阅读功能的话）或给盲人用的感受装置。此外，CSS 还可以与 XHTML、XML 或其他结构文件一起使用，唯一的条件是显示这种文件的浏览器能够接受 CSS 的功能。

5. Flash/Flex 技术

1) Flash

Flash 是 Macromedia 公司（现已被 Adobe 收购）出品的动画制作程序，利用 Flash 可以制作出一种后缀名为 swf 的文件，其中包含了声音、图像和动画等。这种文件既可加入 HTML 中，又可单独作为页面使用。

Flash 的优点是其生成的多媒体文件是矢量图，体积小，还可以边下载边播放，这样避免了用户长时间等待。而且，用 Flash 生成的文件是代码保护的，别人无法看到其源代码，还可以禁止下载。

Flash 使用 ActionScript 编程语言进行编写。ActionScript 是 Macromedia 为 Flash 产品开发的，最初是一种简单的脚本语言，其 3.0 版本是一种完全的面向对象编程语言，语法和 JavaScript 很相似，适用于 Flash 平台各种应用的开发，从简单的动画，到复杂的、富数据的、交互性的各种网页和 RIA 应用程序。

2) Flex

Flex 是 Macromedia 发布的展示服务的新版本，根据 .mxml 文件（纯粹的 XML 描述文件和 ActionScript）产生 .swf 文件，再由 flash player 或者 shockwave player 解释执行，以提供丰富的客户体验，是重要的 RIA 技术。Flex 通过 Java 或者 .NET 等非 Flash 途径解释 .mxml 文件组织组件（component），并生成相应的 .swf 文件。Flex 的 component 和 Flash 的 component 很相似，但有所改进和增强。

运用 Flash 完全可以做到 Flex 的效果。但是，Flash 天生是为了 designer（设计者）设计的，界面和 Flash 的动画概念和编写程序的开发人员格格不入。Flex 是为了吸引更多熟悉 JSP、ASP 和 PHP 等编程的 developer（开发者）而推出的，用更加规范和标准化的 .mxml 描述界面。为了避免理解混乱，从 4.0 版本开始，Flex Builder 更名为 Flash Builder，而 Flex SDK 名称仍然不变，以明确 Flash Builder 是面向设计者的设计工具，而 Flex SDK 是面向开发者的开发语言或者说开发框架。

6. DOM

DOM（Document Object Model，文档对象模型）是 W3C 推荐的处理可扩展标记语言的标准编程接口，主要作用是建立 Web 页面与 Script 或程序语言沟通的桥梁。自从 W3C 建立了 DOM 标准（W3C DOM）之后，以及 DOM 和浏览器兼容之后，DOM 在实际应用中被使用得越来越广泛。

DOM 常用来和 JavaScript 交互，即程序以 JavaScript 编写，但使用 DOM 来存取页面及其元素。这两者之间的结合非常紧密，甚至可以说，如果没有 DOM，人们在使用 JavaScript 时是不可想象的，因为每解析一个节点一个元素都要耗费很多精力，DOM 本身是设计为一种独立的

程序语言，以一致的 API 存取文件的结构表述。

在和 JavaScript 进行交互时，DOM 主要用来解析 XML 文档，当应用程序需要不断地导航、修改文档或随机地一次访问整个文档时，一般就使用 DOM 来解析。在使用 DOM 进行解析时，它在内存中构建起一棵完整的解析树，借此实现对整个 XML 文档的全面、动态访问。也就是说，它的解析是有层次的，即将所有的 HTML 中的元素都解析成树上层次分明的节点，然后可以对这些节点进行增加、删除、修改和查找等操作。

7. JavaScript/AJAX

1) JavaScript

JavaScript 是一种广泛应用于 Web 客户端开发的脚本语言，其源代码以文本格式的字符代码发送给客户端浏览器，发往客户端后由浏览器解释运行。JavaScript 是一种动态、弱类型、基于原型的语言，内置支持类型，其语法类似 Java，一些名称和命名规范也借鉴 Java。常用于为 HTML 页面添加动态功能，操纵 Web 页面上的元素，以实现 Web 页面的客户端交互功能，制作特殊动态效果。例如，注册时用户输入信息的有效性验证、弹出的信息框、鼠标指针的文字跟随、渐隐渐现的图片等丰富 Web 页面表现的各种操作。因此，在 Web 页面设计中得到了广泛应用。

但是，解释语言的弱点是安全性较差。而且，在 JavaScript 中，如果一条语句不被执行，那么后面的语句也无法执行。另外，由于每次重新载入都会重新解释，载入后有些代码还会延迟至运行时才解释，甚至多次解释。因此，在一定程度上速度较慢。

2) AJAX

AJAX (Asynchronous JavaScript and XML，异步 JavaScript 和 XML) 是一种创建交互式 Web 应用的网页开发技术，它有机地利用了一系列相关技术。它的一些部分以前称作动态 HTML 和 远程脚本 (Remote Scripting)。技术上，AJAX 极大地发掘了 Web 浏览器的潜力，开启了大量新的可能性，目前广泛应用于 RIA 应用开发。典型的例子是，Google 在它著名的交互应用程序中使用了异步通信，如 Google 讨论组、Google 地图、Google 搜索建议、Gmail 等。

16.4.4 Web应用系统服务器端技术

Web 服务器端的开发技术也是由静态向动态逐步发展完善起来的。Web 服务器端开发的代码在服务器 (Web 服务器及扩展环境) 端执行，主要有 CGI、ISAPI/NSAPI 等基本开发技术，PHP、ASP、ASP.NET、JSP 等高级开发技术和基于后台数据库的 ODBC、ADO、JDBC 等数据库连接技术。每种技术都有其特点，开发人员要根据需求和具体情况来决定使用哪种系统环境和开发技术。这些技术的主要任务是设计与开发应用程序，在处理用户交互的基础上实现既定的需求，为客户端提供动态生成的数据，实现数据源之间的交互与管理。

1. CGI

CGI (Common Gateway Interface，公共网关接口) 是一种早期应用于 Web 应用的标准，用于 Web 服务器运行服务器端外部程序，称为 CGI 脚本，多用于动态生成 Web 内容。遵循这套接口标准，可以使用任何编程语言来编制 CGI 程序，而不受开发语言的限制，如 C、C++、

VB、Delphi 等。与静态的 Web 获取不同，使用 CGI 可以创建程序，当用户发出请求时，Web 服务器将请求转到目标程序，执行程序后接收其输出，并将输出作为 HTTP 响应发回客户端。CGI 程序是独立的可执行程序，每次收到 CGI 请求时由 Web 服务器调用，服务器进程与 CGI 进程进行进程间通信。用于服务器端 CGI 编程的流行语言有 Perl。

但 CGI 程序的开发效率和运行效率都不高，开发的难度较大，使用不方便，而运行时由于需要单独的进程，因此开销较大。

2. PHP

PHP (PHP: Hypertext Preprocessor, 超级文本预处理语言) 是一种 HTML 内嵌式脚本语言，在服务器端执行。PHP 的语法混合了 C、Java、Perl 以及 PHP 独创的语法。它可以比 CGI 或者 Perl 更快速地执行动态 Web 页面。用 PHP 制作出的动态页面与其他的编程语言相比，执行效率比完全生成 HTML 标记的 CGI 要高许多，PHP 执行引擎还会将用户经常访问的 PHP 程序驻留在内存中，其他用户再一次访问这个程序时就不需要重新编译程序，直接执行内存中的代码即可。PHP 功能强大，几乎所有的 CGI 功能 PHP 都能实现，而且支持几乎所有流行的数据库以及操作系统。PHP 的优势主要表现在以下几方面：

(1) 学习快速。PHP 是一种能快速学习、跨平台、有良好数据库交互能力的开发语言，让 UNIX/Linux 有了一种可以与 ASP 媲美的资本。

(2) 与 Apache 及其他扩展库紧密结合。PHP 可以与 Apache 以静态编译的方式结合起来，而与其他扩展库也可以用这样的方式结合 (Windows 平台除外)。这种方式极大地利用了 CPU 和内存，有效地利用了 Apache 高吞吐能力。同时，外部的扩展也是静态联编，从而达到最快的运行速度。由于与数据库的接口也使用了这样的方式，因此使用本地化调用，这也让数据库发挥了最佳效果。

(3) 良好的安全性。因为 PHP 本身的代码开放，所以其代码在许多工程师手中进行检测，具有了公认的安全性能。此外，PHP 为应用开发也提供了一些内置的安全性支持，典型的如 `mysql_real_escape_string` 函数。

目前有很多基于 UNIX (或 Linux) 服务器的 Web 应用使用 PHP 技术开发而成。

3. ASP/ASP.NET

ASP 和 ASP.NET 是基于 Windows 开发 Web 应用的重要且被广泛使用的技术。

1) ASP

ASP (Active Server Page, 动态服务器页面) 是微软开发的代替 CGI 脚本程序的一种嵌入式网页中的脚本，是可由服务器执行的服务器端脚本技术。ASP 可以与数据库和其他程序进行交互，是一种简单、方便的编程技术框架。微软把 ASP 描述为：“一个服务器的脚本环境，在这里可以生成和运行动态的、交互的、高性能的 Web 服务器应用程序”。ASP 的主要特性是能够把 HTML、脚本和组件等有机地组合在一起，形成一个能够在服务器上运行的应用程序，并按用户要求专门制作的标准 HTML 页面送给客户端浏览器。

ASP 的主要特性表现如下：

(1) 利用 ASP 可以实现突破静态页面的一些功能限制，实现动态页面技术。

(2) ASP 文件包含在 HTML 代码所组成的文件中, 易于修改和测试。

(3) 服务器端的 ASP 解释程序会在服务器端执行 ASP 程序, 并将结果以 HTML 格式传送到客户端浏览器上。因此, 使用各种浏览器都可以正常浏览 ASP 所产生的页面。

(4) ASP 提供了一些内置对象, 使用这些对象可以使服务器端脚本功能更强。例如, 可以从 Web 浏览器中获取用户通过 HTML 表单提交的信息, 并在脚本中对这些信息进行处理, 然后向 Web 浏览器发送信息。

(5) ASP 可以使用服务端 ActiveX 组件来执行各种各样的任务, 如访问数据库和文件系统等。

(6) 由于服务器是将 ASP 程序执行的结果以 HTML 格式传回客户端浏览器, 因此使用者不会看到 ASP 所编写的原始代码, 可防止 ASP 程序被窃取。

2) ASP.NET

ASP.NET 是用于动态 Web 应用构建的免费 Web 框架, 包括很多开发 Web 应用所需的服务, 作为 .NET Framework 的一部分提供, 可以使用与公用语言运行时 (CLR) 兼容的任何语言编写应用程序。使用这些语言可以利用 CLR、类型安全和集成等优点。编写的代码在服务器端首次运行时进行编译, 在服务器上运行。ASP.NET 作为 Web 服务器的一部分, 处理浏览器发出的请求, 处理已经映射到其上的文件扩展名, 如 .aspx、.ascx、.ashx 和 .asmx。使用 ASP.NET, 一般以 IIS 作为服务器以提供完整的 Web 服务器功能。

通用语言的基本库、消息机制和数据接口的处理都能无缝整合到 ASP.NET 的 Web 应用中。程序员可以选择一种或多种最适合的, 只要是兼容 .NETCLR 所支持的语言来编写程序, 包括 C#、Visual Basic、JScript.NET、managed C++、J#, 其中 C# 在近些年更为常用。将来, 多种程序语言协同工作的能力能使程序员现在基于 COM+ 开发的程序完整地向 ASP.NET 移植。

4. Servlet

Servlet 是一种由 Java 编写的服务器端程序, 可以动态生成 Web 页面, 是 Java EE 的一部分。Servlet 运行于支持 Java 的 Servlet 容器中, 如 Java EE 的 Servlet 容器, 在被调用后会被动态地载入容器, 由容器解释执行。Servlet 的 .java 文件先编译成 .class 文件, 然后部署在 Servlet 容器。

Servlet 支持 Request/Response 模型, 在服务器端接收 Web 客户端的请求并给出响应, 一般通过 HTTP 协议来完成。Servlet 容器提供了侦听请求的服务, 能把客户请求和响应信息包装在特殊的 Request/Response 对象中, 交给 Servlet 处理, 然后由 Servlet 处理请求并通过 HTTP 协议将响应通过容器转发到客户端。

在基于 MVC 模式的 Web 应用中, Servlet 充当控制器的角色, 用来处理 HTTP 请求, 管理应用的工作流程。Servlet 通过用户所发送的 HTTP 请求, 接收用户的输入事件, 并把这些信号翻译成消息传递给封装了请求业务逻辑的 JavaBean 或 EJB 进行交互, 最后激活 JSP。

Servlet 的主要优势如下:

(1) Java 语言的优点。由于使用 Java 语言开发, 具有 Java 语言的所有特性, 如可移植性、内存自动回收、良好的面向对象特性、异常处理机制、包括联网支持在内的大量应用程序接口和易用性等内容。

(2) 执行效率高。一个 Servlet 可以同时处理多个请求，每个请求将生成一个新的线程，多个客户能够在同一个进程中同时得到服务。

(3) 构造的控制器功能强。Servlet 是模块化的，每个模块执行一个特定的任务，也可以协同工作，构成功能更强的“Servlet 链”。

5. JSP

JSP (Java Server Page, Java 服务器页面) 是一种将 Java 代码嵌入 HTML 页面中实现的服务器端 Web 开发技术，在服务器端进行解析，动态生成页面传递给客户端，使 Web 应用开发人员可以使用 HTML 和 XML 模板以及 Java 代码在服务器上建立动态内容。同时，它还是一种安全、快速并且与服务器平台无关的设计方法。JSP 是 Java EE 的一部分，本质上是一种高层的 Servlet，从使用的角度而言，与其他页面编写脚本有很大的相似性。

在 JSP 中，HTML 代码主要负责描述信息的显示，而“<%”和“%>”之间的 Java 程序代码则用来描述处理逻辑。JSP 页面在服务器端进行初次请求时转换成 Servlet，然后编译成字节码进行执行，并将执行结果重新嵌入 HTML 代码中发送给浏览器。客户端浏览器不需要任何附加的软件支持。

JSP 作为 Java EE 的部分，可以和 JavaBean、Servlet 或 EJB 结合使用。JSP 的优点突出，具体包括：

- (1) 使内容的生成和显示分离；
- (2) 生成可重用的组件；
- (3) 采用标签简化页面开发，通过开发定制标签库，实现了 JSP 技术的扩展；
- (4) 具有 Java 的优点，包括健壮存储管理和安全性，可靠且移植方便等；
- (5) 企业产品多样性，在 Java EE 平台内容中不仅包括管理复杂企业的应用程序，而且包括事务管理技术和 pooling 资源管理技术。

6. Perl/Ruby/Python

1) Perl

Perl 是一种自由且功能强大的脚本语言，最初的设计者为 Larry Wall。Perl 借取了 C、sed、awk、shell scripting 以及很多其他编程语言的特性。其中，最重要的特性是它内部集成了正则表达式和巨大的第三方代码库 CPAN。从最初被当作一种在跨平台环境中书写可移植工具的高级语言开始，Perl 就已经被广泛地认为是一种工业级的强大工具，可以在任何地方用来完成工作。

Perl 逐步发展成为一种功能齐全的程序设计语言，特别是在各种计算平台上，它被用作 Web 编程、数据库处理、XML 处理以及系统管理。它能够完成所有这些工作，同时仍然是处理小的日常工作的完美工具，这是它的设计初衷。Perl 快速而且特别有用。随着 Perl 的日益流行，Perl 实际上已经被所有 UNIX（包括 Linux）捆绑在一起作为标准部件发布，而且也被广泛用于 Windows 和几乎所有其他的操作系统。Amiga、BeOS、VMS、MVS 和 Apple Macintosh 等也只是 Perl 已经完成移植的平台的一小部分。

2) Ruby

Ruby 是 20 世纪 90 年代开发出的一种面向对象、解释型脚本语言，综合了 Perl、Python 和

Java 等语言的特点, 语法简单, 擅长文本处理和系统管理等任务, 还有异常处理以及迭代器等构造, 这些特性使编程简单明了。Ruby on Rails (RoR) Web 应用框架出现后, Ruby 即被广泛应用于 Web 应用的开发中。

3) Python

Python 是一种面向对象、解释型计算机程序设计语言, 也是一种功能强大而完善的通用型语言。它具有非常简捷而清晰的语法特点, 适合完成各种高层任务, 几乎可以在所有的操作系统中运行。Python 支持最新的 XML 技术, 可用于 Web 应用的开发。

Python 具有很多优秀脚本语言的特点: 解释的、面向对象的、内建的高级数据结构、支持模块和包、支持多种平台、可扩展, 还支持交互式或图形方式运行, 拥有众多的编程界面支持各种操作系统平台以及众多的各类函数库。与其他脚本语言相比, Python 程序员可以借助 Python 语言提供的 API, 使用 C 或者 C++ 来对 Python 进行功能性扩展, 从而既可以利用 Python 方便灵活的语法和功能, 又可以获得与 C 或者 C++ 几乎相同的执行性能。同时, Python 通过与 C 语言的有机结合有效解决了脚本语言执行慢的问题, 从而使脚本语言的应用范围得到了很大扩展。

Python 在服务器端的编程可以通过 Web 应用服务器端开发框架加以支持, 也可以通过 CGI 脚本编程。另外, 也有使用 Python 的服务器和内容管理系统的实现。典型应用如 Google 的搜索引擎大量使用 Python; Zope 应用服务器和著名的 BT 下载工具 Bit Torrent 由 Python 编写。

事实上, Python 也可以用于客户端编程。Web 浏览器中的插件提供对 DOM 模型的支持, 也可以使用插件或转换成 JavaScript 等方式来支持 Python, 然后在页面中使用 `<script type="text/python">` 来替换 JavaScript 的使用方式 `<script language="javascript">` 即可。

16.4.5 Web应用系统部署

Web 应用的部署是把已经构建完成的 Web 应用发布到服务器上, 通过对 Web 应用的运行环境进行配置, 使得用户可以通过 Internet 访问该 Web 应用。Web 应用的部署一般包括打包、发布和评估。

1. 部署粒度

因为 Web 应用的开发是增量迭代进行的, 所以部署也会在 Web 应用发布的过程中发生很多次。

Web 应用发布以一种非常细粒度的方式完成, 在测试完某个组件后, 将新组件从预发布服务器再发布到运行服务器。例如, 发布对内容或功能的更正、演化列表中重要新条目, 或对已经部署了的 Web 应用的用户“现在”就要求的新内容或功能。

在很多情况下, 这种细粒度的发布方式可能并不合适。如果有很多不断发生的小变化, 用户可能会感到困惑, 而且集成错误的可能性和副作用也会增加。因此, 除非是一个变化更正一个破坏性的错误时会进行这种细粒度的发布, 其他在更新或变化影响不大的情况下, 最好把一组变化打包进行发布。

每个打包发布周期都为最终用户提供了一个具有可用功能和特性的 Web 应用增量。每个

评估周期都为 Web 应用团队提供重要指导，并为下一个增量做出内容、功能、特征和方法的修改。

Web 应用的部署不是一气呵成的，也需要经历一个渐进的过程。Web 应用不断升级的特点，使 Web 应用升级总是伴随着新旧版本兼容问题的风险，用户使用习惯突然改变而造成用户流失的风险，系统宕机的风险。为了避免这些风险，Web 应用的部署可以采用灰度发布的策略，其主要思想是把影响集中到一个点，然后再发散到一个面，出现意外情况后很容易就回退，然后把收集到的用户反馈作为 Web 应用修改的依据对 Web 应用需要修改的地方进行改进。

2. 部署原则

一个 Web 应用增量的部署代表 Web 工程项目的的一个重要里程碑。一个 Web 应用的部署为最终用户提供了内容和功能以及立竿见影的收益，但是如果部署的计划不好、错误很多，而且执行效率不高，那么用户会很不满意。为了确保 Web 应用的顺利部署，Web 应用项目团队在准备交付一个增量时，应该遵循如下一些关键原则：

(1) 管理客户对 Web 应用增量的期望。管理客户期望的首要问题是与客户进行有效沟通，客户通常并不希望看到团队承诺交付而又没有实现的内容，Web 应用项目团队不应该轻易对客户做出很可能完不成的承诺，敏捷方法将便于内容和功能的增量交付，为客户提供 Web 应用更早的反馈，因此支持对客户期望的管理。

(2) 安装与测试交付包。在部署前，需要在不同的硬件平台、不同的浏览器配置和网络带宽以及不同的安全性设置下进行完整的测试。

(3) 交付前建立支持制度。用户往往期望在出现问题时得到及时的反应和精确的信息，所以应该计划支持，准备支持过程和响应，并且建立合适的记录保持机制，这样 Web 应用团队才能够对支持请求的种类进行分类评估。

(4) 先改正有缺陷的 Web 应用，然后再交付。出于时间的考虑，一些 Web 应用开发人员会交付一些质量很低下的增量，这样是不妥的。Web 应用一方面为用户带来了巨大的便利性，提高了用户的工作效率，另一方面也为 Web 应用项目团队提供一些有用的反馈。当一个增量投入使用后，应该鼓励最终用户对 Web 应用的特征、功能等方面的内容进行客观的评价，Web 应用开发人员收集并记录这些信息，及时调整自身的开发计划以及改正 Web 应用中存在的问题与隐患，进而更高效地开发出用户期望的产品。

例如，某电子商务公司的主要业务是书籍、服装、家电和日用品的在线销售。随着公司业务发展和用户规模不断扩大，现有的网上交易系统无法正常处理日益增大的请求流量，公司决策层决定升级其网上交易系统。在确定使用基于负载均衡集群的系统升级方法后，李工给出了一个基于 LVS (Linux Virtual Server) 的负载均衡集群实现方案。公司的系统分析师在对现有系统进行深入分析的基础上，认为以下两个实际情况对升级方案影响较大，需要对该方案进行改进：

(1) 系统需要为在线购物提供购物车功能，用来临时存放选中的产品。

(2) 系统需要保证向所有的 VIP 用户提供高质量的服务。

在实际应用中，通常会将传输层负载均衡方法与应用层负载均衡方法结合起来使用，以提

高系统整体的性能、可用性和可靠性。第一种情况的描述说明系统需要提供应用会话数据支持。通常采用会话服务器（Session Server）机制在服务器端存放应用会话数据。但需要注意的是，应用会话数据大多数情况下是不可恢复的，因此采用支持应用会话数据容错的解决方案非常重要。第二种情况的描述要求保证特定用户的服务质量。当数据量不断增长时，由于在会话服务器或缓存服务器，以及业务服务器和会话（或缓存）服务器之间可能会反复移动较大量的数据，从而无法保证服务质量。因此，通常的做法是基于应用层负载均衡器实现客户端联系（Client Affinity），将某个客户端的所有请求转发到相同的服务器上，使得一个用户会话的所有请求被同一服务器实例处理，从而保证服务质量。

3. 部署环境

Web 应用发布的软件系统主要包括操作系统软件和 Web 应用服务器软件两方面的内容。目前，几乎所有的操作系统都可以支持 Web 应用的发布，常见的用于 Web 应用发布的操作系统包括 Windows Server 系列、Linux 和各类 UNIX 等。Windows 家族产品中的其他操作系统如 Windows XP 也支持一定的 Web 应用发布能力，但一般只用作平时的开发测试及学习。可进行 Web 应用发布的服务器软件有 Apache、IIS、WebSphere 及 WebLogic 等。其中，除 IIS 只能适用于 Windows 操作系统外，其他软件都可跨平台使用。

应用服务器在基于 Web 的数据库应用系统中已经得到了广泛的应用。如今，各大主要软件厂商纷纷将应用服务器作为其电子商务平台的基础。由于应用服务器本身是一个正在不断发展的概念，不同的产品之间有很大的差别，但是其核心结构，以及需要解决的主要问题都是相近的，区别仅在于各个产品解决的具体方法不同。下面是应用服务器需要共同解决的部分问题：

（1）负载均衡。应用服务器实现负载均衡的方法很多，如在应用服务器本身的实现上，有基于进程的方式和基于线程的方式。

（2）数据库连接池。在应用服务器系统中，一般都会采用数据库连接池的技术。

（3）高速缓存机制。为了提高性能，许多应用服务器都采用了高速缓存机制。

例如，某公司希望实现一个电子商务系统，要求该系统必须符合工业标准且支持多种操作平台。也就是欲选择一种应用服务器产品，该产品符合工业标准，支持多种操作平台，适合电子商务应用。要想做出正确的选择，必须能够对以下几个要求有正确的理解：

（1）符合工业标准，主要是指应用服务器中采用的中间件技术符合工业标准，如 J2EE、CORBA、XML 等。

（2）支持多种操作系统，也就是能够在 Windows 系列、UNIX 系列等多种操作系统环境中使用，基于 CORBA 和 J2EE 的都能够满足这个要求。而微软的 MTS 则只能够运用于 Windows 系列操作系统平台，显然无法满足这个要求。

（3）适合电子商务应用，根据电子商务的应用特点，要求应用服务器能够提供诸如事务管理、组建容器等一系列适合分布式应用的技术。另外，还应该具有高扩展性、伸缩性的特点，以满足电子商务发展的业务扩展需要。

根据上面的分析，结合各种主流应用服务器的特色，可以发现应用服务器产品中符合这些要求的比较多。比较适合的选择包括 BEA 的 WebLogic、IBM 的 WebSphere、Sun 和 Netscape

联合推出的产品 iPlanet Application Server 等。当然，类似 JBoss 这样的应用服务器也是正确的选择。

也可以选择 J2EE 应用服务器（如 WebLogic、Websphere 等），它支持多种操作系统，如 Windows、UNIX、Linux 等，同时也是工业标准。具体分析如下：

（1）数据库操作支持。JDBC 数据源，通过缓冲数据库连接，提供高效、可靠的数据库操作。

（2）安全性控制。J2EE 提供声明性安全控制，用户在部署描述符中通过声明的方式来控制应用系统的安全性。例如，可以控制如何进行身份认证，控制哪些角色可以访问哪些资源或执行哪些操作等。

（3）事务控制。J2EE 应用服务器支持将事务控制交给容器自动管理，或者利用 JTA 在代码中自己控制事务。

（4）与其他系统交互。基于 JCA 或者 JMS，另外可以直接访问 CORBA 构件。

4. 版本控制和 CMS

版本控制和内容管理系统（CMS）在 Web 应用构建和部署活动中起着重要的作用。由于 Web 应用变更快且频繁，因此要使 Web 应用构建和部署成功，就必须在变更管理工具的辅助下管理好变更。版本控制和 CMS 的优点如下：

（1）版本控制和内容管理系统都是变更管理工具，而且和其他内容一致。

（2）确保所发布的内容正确，而且和其他内容一致。

（3）控制和跟踪内容的变更，包括对谁可以做出变更进行强制的实现机制。

（4）检验实现了一个功能的正确版本及其和相关功能版本的正确对应。

（5）使 Web 工程团队在系统失败或崩溃时，能快速重建 Web 应用。

（6）允许团队在最新版本遇到严重的、未预见的错误时，回滚到前一个版本。

随着 Web 应用的规模和复杂性的增加，构建和部署的每一步的全面版本控制和健壮 CMS 的要求也在增加。

16.5 Web 应用系统测试

Web 应用测试是保障 Web 应用高可靠性和高可用性的必要手段，随着需求量与应用领域的不断扩大，对 Web 应用的正确性、有效性、可用性和对 Web 服务器等方面都提出了越来越高的要求，因此如何对 Web 应用进行有效的、系统的测试已经成为了 Web 应用开发的重要工作之一。Web 应用的测试、确认和验收是一项重要并且富有挑战性的工作，它对测试人员也提出了更高的要求。Web 应用测试不但需要检查和验证 Web 应用是否按照设计的要求运行，而且还要测试 Web 应用在不同客户端的浏览器上运行是否正确、显示是否合适、Web 应用是否符合用户的使用习惯、Web 应用是否能够满足并发量的要求等内容。更重要的是，还要从最终用户的角度进行安全性和可用性等方面的测试。然而，Internet 和 Web 媒体的不可预见性使得测试 Web 应用变得非常困难。因此，必须制作详细的测试计划，设计切合实际的 Web 应用测试规程、认

真仔细地执行测试操作才能最大限度地发挥 Web 应用测试的效能,尽可能地发现 Web 应用中存在的问题,最大限度地提高 Web 应用的质量。

根据测试内容的不同,Web 应用测试主要包括功能测试、内容测试、性能测试、Web 页面测试、客户端兼容性测试、安全性测试等内容。

16.5.1 Web应用测试概述

Web 应用测试与传统软件测试一样,主要目的是发现错误和缺陷。Web 应用具有多层体系结构,客户、数据通信、硬件以及服务器之间依赖关系非常复杂,每层内以及各层之间都有可能发生故障。例如,客户端浏览器的版本、型号不同,所采用的页面渲染技术也不同,使得有些信息往往不能正常显示,从而产生兼容性问题和显示故障;服务器、数据库的负载能力有限,在用户访问达到高峰时,响应时间太长甚至不接受用户的访问等。要发现、分析进而排除这些故障,通常需要进行多方面的测试,以保证 Web 应用具有良好的功能、性能、兼容性和安全性。

从测试角度看,与传统的软件测试相比,Web 应用测试的特性和面临的挑战主要表现在以下方面:

(1) 用户数量巨大,并要求能提供对 Web 资源的跨平台全局访问,需要有处理并发事务的能力,因而需要进行多用户访问的性能测试。

(2) “内容”中的错误通常只能靠人工完成;超文本结构中页面之间的链接关系要确保正确是一个很大的挑战;展示层的“软”需求(如艺术性)难以测试;全球性使得 Web 应用的可用性和多语言方面的测试难度更大。

(3) Web 应用一般采用多层架构,因此需要对测试结果进行综合分析,以确定系统功能或性能缺陷存在的具体位置。

(4) Web 应用中常常集成不同的软件,如 Web 服务器、数据库、中间件服务器等,以及第三方软件和框架,Web 应用的质量受所有这些第三方软件和组件及其交互的质量的影响,测试也必须包括对第三方软件和组件及其集成和配置的测试。

(5) Web 浏览器提供的导航,如回退(back)、前进(forward)和刷新(refresh)等按钮,也会经常引发各种错误。

(6) 实现 Web 应用的不同种类技术的出现,使得以适当的粒度定义和验证 Web 测试模型的各个组件变得非常困难,充分建立组件间的关系也是一个大的难题。

(7) 从运行机制上看,Web 应用具有分布式、动态性、多平台、交互式 and 超文本等特点,运行环境异构、自治,要求针对其特性分别选择测试方法。为降低测试成本,提高测试效率,需要一定的辅助工具以提高测试执行的自动化水平。而 Web 应用测试方法和工具的不成熟性,使得测试面临错误、难以使用等问题。

Web 应用测试的诸多特性,使得不能仅用传统的方式来对 Web 应用进行测试。在 Web 应用整个生命周期的各个阶段,测试的侧重点有所不同。设计阶段测试的主要任务是:估算服务器端容量的规划是否合理,系统的安全设计是否合理,数据库设计是否合理,检查客户端设计的功能是否正确合理,检查系统的网络拓扑结构、容量设计是否合理。开发阶段测试的主要任

务是：代码测试及组件测试，检查设计的代码能否满足规格需求。运行阶段测试的主要任务是：功能测试、性能测试、安全性测试、配置测试、兼容性测试及易用性测试。维护阶段测试的主要任务是：根据维护的内容实施开发及运行阶段中的各个相关方面的测试。所有这些方面测试的实施，对保证 Web 应用的质量及可靠性至关重要。

Web 应用的开发通常采用敏捷开发方式，可以不断调整以支持用户和客户多变的需求，并能够尽快投入市场。但这给测试带来了很大的压力：一旦 Web 应用发生了变化，必须进行回归测试，以便确定修改是否达到了预期的目的、检查修改是否损害了原有的正常功能；同时还需要补充新的测试用例来测试新的或被修改了的功能。

16.5.2 Web应用测试过程

传统的测试方法将测试分为单元测试、集成测试、系统测试和验收测试，而过程包括计划、准备、执行和生成测试报告。传统的测试方法定义工作结果，如质量计划、测试测量、测试计划、测试用例、测试度量、测试环境和测试报告等；还定义测试角色，如测试经理、测试顾问、测试专家、工具专家等；还定义创建工作结果的详细步骤，如分析可用测试数据、准备或提供测试数据。

敏捷方法定义质量目标，然后根据团队自组织创建软件以满足这些质量目标。敏捷方法中开发团队协作和自治地寻找对问题的解决方案，测试不只是关于角色的，而是团队中紧密协作和可用能力的最佳使用。这说明，测试是一个集成开发活动，整个团队共同为质量负责，也因此共同进行测试。

对于 Web 应用测试而言，由于其开发周期短，虽然也需要完成这些不同层次的测试和执行测试过程，但并非在时间上顺序执行，而更倾向于尽早开始测试，迭代地测试小的增量，而且通常选择一些最重要的工作结果。

Web 应用测试更多采用敏捷开发方法。敏捷方法中单元测试用例由开发人员完成，其他测试工作由非开发人员完成。这就要求开发人员和测试人员之间需要有效的沟通和协调、相互理解，一旦有功能出现变化，这种变化可以立即被发现。另外，反馈、自动测试是短的开发周期和重构的前提。极限编程（XP）中结对编程、客户在场、持续集成和测试先行对 Web 应用测试和质量保证影响尤其大。特征驱动开发（FDD）并不使用结对编程，但是强调代码评审。但是，这两种方法都确保在编码开始时就采用静态质量保证技术。

一些研究者给出了 Web 应用测试的一般过程，具体包括：功能测试、内容测试和评审、Web 页面测试、导航测试、接口测试、配置测试、安全测试、性能测试等。测试主要包括以下几个步骤：

- （1）首先需要对被测的 Web 应用进行需求分析，即对所做的测试做一个简要的介绍，包括描述测试的目标和范围，所测试的目标需要实现一个什么样的功能，总结基本文档、主要活动。

- （2）定义测试策略和方法，这里包括测试开始的条件、测试的类型、测试开始的标准以及所测试的功能、测试通过或失败的标准、结束测试的条件，测试过程中遇到什么样的情况终止和怎么处理等。

(3) 确定测试环境的要求(包括软件和硬件方面),选择合适的测试用例、选择合适的测试工具、决定执行测试的人员,以及确定测试要做到何种程度(测试的充分性)。

(4) 针对测试的行为,描述测试的细节,包括测试用例列表、进度表、错误等级分析、对测试计划的总结、测试过程会出现的风险分析等。

Web应用的测试,首先需要测试最终用户能够看到的内容和界面功能,随着测试的深入,再测试信息和功能架构以及导航方面的内容,最后测试的内容转到用户并不总是可见的测试技术能力,即Web应用基础设施及其安装与实现的问题。

Web页面测试验证用户界面的交互机制以及美学方面的内容,目的是发现由于实现了糟糕的交互机制而导致的错误,或者由于不小心而导致的遗漏、不一致或歧义性。

性能测试包括一系列测试,设计这些测试用来评估Web应用的响应时间及可靠性如何受增长的用户通信量和功能复杂性的影响,用来查找哪些Web应用组件与性能下降有关等内容。

安全性测试是将一系列测试合并起来,攻击Web应用及其环境中的弱点,目的是发现可能的安全漏洞。

本节将主要论述Web系统测试中几个比较重要的方面,并详细分析其具体的操作内容。

16.5.3 Web应用功能测试

功能测试在整个测试过程中起着至关重要的作用,它结合Web应用规格说明的要求,保证Web应用在功能上能够达到预期的目标。只有满足功能需求的Web应用才可能是客户需要的,一个不满足功能要求的Web应用可以说是无用的Web应用。根据测试内容的不同,功能测试主要可以分为链接测试、表单测试、数据校验、Cookie测试和数据库测试。

1. 链接测试

链接测试是Web应用所特有的测试,链接测试可分为三方面的内容:首先,所有链接是否按指示的那样确实链接到了该链接所指向的资源;其次,所链接的资源是否存在;最后,是否所有页面都能够被链接到,不存在孤立页面(所谓孤立页面是指没有链接指向该页面,只有知道正确的URL地址才能访问)。

链接测试需要对整个Web应用的所有链接进行,而一般的Web应用内的链接错乱复杂,犹如一张大蜘蛛网,稍有疏忽便有测试不完全的地方。因此,引入链接自动化测试能够大幅提高链接测试的效率。现在已经有许多工具可以帮助测试人员进行链接测试,如Xenu Link Sleuth和HTML Link Validator。链接测试必须在集成测试阶段完成。也就是说,在整个Web应用(或增量)的所有页面开发完成之后进行链接测试。

2. 表单测试

当包含表单元素的页面呈现给用户时,用户填写相关内容并提交到服务器,然后再由服务器端程序动态生成页面返回给用户,从而实现一次完整的交互过程。当用户使用表单元素进行各种操作时,必须校验用户提交给服务器的信息的正确性和有效性。例如,用户填写的出生日期与职业是否恰当,填写的所属省份与所在城市是否匹配等。如果使用了默认值,还要检验默认值的正确性。如果表单元素只能接受指定的某些值,则也要进行测试。例如,只能接受某些

字符，测试时可以跳过这些字符，看是否会出现错误。也就是说，要测试这些程序，需要有数据正确性验证，对异常处理的验证，还需要验证服务器能正确解析和使用这些信息。

交互测试一般要确保以下几方面的内容：

- (1) 对表单元素中的标识域给出正确标记，并且为用户显示地标识出强制域。
- (2) 对每项用户输入进行正确性和合法性检查。
- (3) 对用户输入错误时的异常处理机制进行检查。
- (4) 用户没有从下拉菜单或按钮组中进行选择时，使用合适的默认项。
- (5) 浏览器“后退”等功能没有破坏输入到表单中的数据。

在更具体的层次上，测试应该确保：①表单元素有合适的宽度和数据类型；②表单元素建立了合适的安全措施，防止用户输入的文本字符串长度大于某个预先定义的最大值；③对下拉菜单中所有合适的选项进行详细说明，并按照对最终用户有意义的方式排序；④浏览器“自动填充”特性不会导致数据输入错误；⑤ Tab 键或一些其他键能够使输入焦点在表单元素之间正确移动。

3. 数据校验

如果根据业务规则需要对用户输入进行一些正确性和合法性校验，需要保证这些校验功能正常工作。例如，省份的字段可以用一个有效列表进行校验。在这种情况下，需要验证列表完整而且程序正确调用了该列表。例如，在列表中添加一个测试值，确定系统能够接受这个测试值。

在测试用户交互时，该项测试和交互测试可能会有一些重复。

4. Cookies 测试

Cookies 通常用来存储用户信息和用户在 Web 应用中的操作。当一个用户使用 Cookies 访问了某一个 Web 应用时，Web 服务器将发送关于这一用户的信息，并把该信息以 Cookies 的形式存储在客户端计算机上，这可以用来创建动态和自定义 Web 页面或者存储登录内容等信息。

如果 Web 应用使用了 Cookies，就必须检查 Cookies 是否能正常工作。测试的内容可包括 Cookies 是否起作用，是否按预定的时间进行保存，刷新 Web 页面对 Cookies 是否有影响、有什么影响等。如果在 Cookie 中保存了注册信息，请确认该 Cookie 能够正常工作，而且已对这些信息加密。如果使用 Cookies 来统计次数，需要验证次数累计正确。关于 Cookies 的使用可以参考浏览器的帮助信息。

可以通过采用 BrowserHistoryView 等测试工具进行该项测试。BrowserHistoryView 是一个搜寻并显示出计算机中所有的 Cookies 档案的工具，它记录了是哪一个 Web 应用写入 Cookies 的，内容有什么，写入的时间日期及此 Cookies 的有效期限等信息。

5. 数据库测试

数据库测试包括测试实际内容及其完整性，以确保数据没有损坏且模式正确。从功能角度而言，在使用了数据库的 Web 应用中，一般可能发生两种错误：数据完整性错误和输出错误。数据完整性错误主要是由于用户提交的表单信息不正确而造成的，数据完整性错误是使错误的

结果被保存下来或者使字段、记录、表和数据库中数据失效的任何错误。例如，可能漏掉了表中的记录，或者没有正确更新而导致数据过期等。而输出错误主要是由在数据提取和操作数据指令过程中发生的错误引起的。针对这两种情况，可分别进行测试。

对数据库功能的测试可以依赖工具进行，常用的数据库测试工具有 DBunit、QTP、DataFactory、Crash-me（MySQL 自带的测试数据库性能的工具）等。

16.5.4 Web应用性能测试

由于 Web 应用无法确定客户端用户对 Web 应用的访问情况，用户数目会处于不断变化之中，而 Web 服务器的承载能力有限，并发数据量过大甚至超过服务器的承载能力，会导致 Web 应用阻塞甚至崩溃。

1. 性能测试内容

遇到关于性能测试的相关问题时，主要考虑的 Web 应用性能测试可以划分为速度测试、负载测试、压力测试、强度测试、并发测试、大数据量测试、配置测试和可靠性测试等。

1) 速度测试

速度测试包括网络连接速度测试和业务处理速度测试。

用户连接到 Web 应用的速度根据上网方式和带宽的变化而变化。当下载一个程序时，用户可能需要等待比较长的时间，但如果只是访问一个页面，Web 应用响应时间太长（如超过 3s），就可能导致用户没有耐心等待而离开，所以要对 Web 应用进行连接速度测试。

业务处理速度测试目前主要是针对关键业务进行手工测速，可以在多次测试的基础上求平均值，可以与测试工具测得的响应时间等指标作对比分析。

2) 负载测试

负载测试主要是确定在用户可接受的响应时间内，系统能够承担的并发用户的数量。由于成千上万的用户可能在同一时刻访问同一个 Web 应用，为了保证系统的正常运行，必须测试在最重的工作负载下 Web 应用的运行情况。负载级别可以是某个时刻同时访问 Web 应用的用户数量，也可以是在线数据处理的数量。例如，Web 应用能允许多少个用户同时在线，如果超过了这个数量，Web 应用是否能承受，会出现什么现象，Web 应用能否处理大量用户对同一个页面的请求等。

可以通过脚本来生成成千上万的“虚拟用户”同时访问 Web 应用并与 Web 应用进行交互。这些虚拟用户执行各种典型任务，如浏览 Web 页面、购买商品、提交数据、搜索数据库等。在虚拟用户执行这些任务的同时，记录下服务器的响应时间。当测试执行完成以后，分析通过负载测试得到的数据，如在不同交互情况下的 Web 页面传送所需时间、Web 页面传送出错信息等，经过一定的分析、计算、处理，得出 Web 应用能同时支持的用户数目、交互数目等，并尽可能找出多用户并发访问的瓶颈，最后以报告和图表的形式显示测试情况下 Web 应用的执行情况以及潜在问题存在的方

多用户访问的瓶颈可能发生在 Web 服务器、Web 应用服务器或（和）数据库服务器上，要明确确定瓶颈所在并非易事。另外，要确定同时访问 Web 应用的用户数目并不容易。可以参考

系统给出的参数结合实际情况进行估计,也可以选择有代表性的分布算法(如指数分布、常量分布、泊松分布等算法)来估计。

图 16-8 显示了经统计而得到的 Web 应用负载、Web 应用响应时间与用户可能放弃的比例之间的关系。从图中可以看出,并发用户数目越多,Web 应用的响应时间越长,从而用户放弃执行该动作的可能性越大。因此,必须从 Web 应用的实际负载能力出发,确定合适的并发用户数目,以先保证服务质量,然后在系统性能稳定的范围内,再考虑为更多的用户提供优质服务。

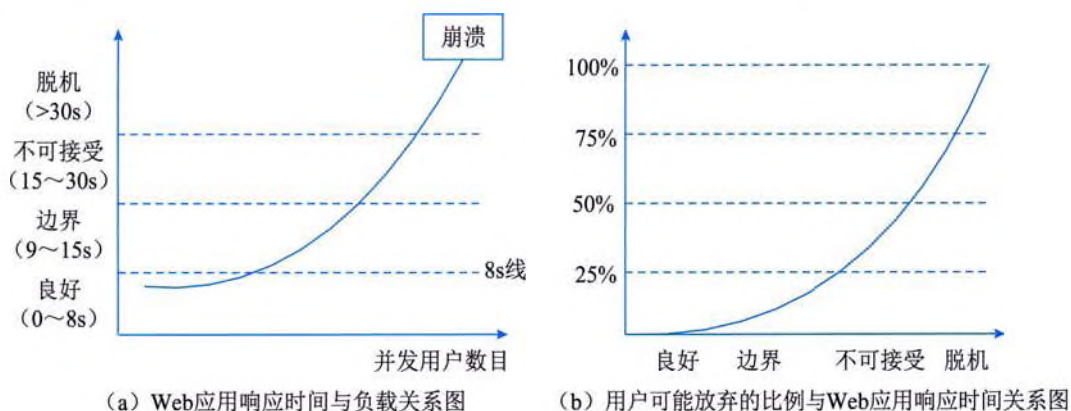


图 16-8 多用户性能测试中部分数据关系图

负载测试应该安排在 Web 应用部署以后,在实际的网络环境中进行测试。因为一个企业内部员工,特别是项目组人员总是有限的,而一个 Web 应用能同时处理的请求数量将远远超出这个限度。所以,只有部署之后,接受负载测试,其结果才是正确可信的。

3) 压力测试

压力测试是负载测试的延续,通过对 Web 应用不断加压,来发现其在什么条件下变得不可承受,查出 Web 应用对异常情况的抵抗能力,找出性能瓶颈,从而获得系统能提供的最大服务级别的测试。压力测试关注 Web 应用在大量并发用户、传送大量数据和大业务量的情况下的性能变化,关注 Web 应用能否长时间运行,响应是否太慢、系统是否崩溃、能否恢复等。

在很多情况下,可能会有黑客或不法分子试图通过发送大量数据包来攻击服务器。出于安全的原因,测试人员应该知道当系统过载时,需要采取哪些措施,而不是简单地提升系统性能。

压力测试的主要手段是通过产生模拟业务对被测试系统进行加压,具体如下:

- (1) 当正常的用户点击率为“100/s”时,运行点击率为“500/s”的测试用例。
- (2) 定量地增长数据输入率,检测对数据处理的反应能力。
- (3) 运行需要最大存储空间(或其他资源)的测试用例。
- (4) 运行可能导致操作系统崩溃或此版数据剧烈抖动的测试用例。

一个好的压力测试必须能够提供产生压力的手段、能够对后台系统进行监控、能够对压力数据进行分析,并快速找出被测系统的瓶颈。当前流行的压力测试工具有 LoadRunner、WAS 和 WebLoad 等。

4) 强度测试

强度测试主要用于检查程序对异常情况的抵抗能力,检查系统在极限状态下运行时性能下降的幅度是否在允许的范围内。强度测试可以测试系统在超负荷情况下的表现,也可以测试系统在最差工作环境下的工作能力,或者验证系统在标准工作压力下的各种资源的最下限指标。

5) 并发测试

并发测试主要是指当测试多个用户同时访问同一个 Web 应用、同一个模块数据记录时是否存在线程同步问题、死锁或其他性能问题。其目的主要体现在:以真实的业务为依据,选择有代表性的、关键的业务操作设计测试案例,以评价系统的当前并发能力。

在实际测试工作中一般都借助工具来模拟并发访问,如 LoadRunner。

6) 大数据量测试

大数据量测试主要测试运行数据量较大时或历史数据量较大时的性能情况,一般针对某些特殊的核心业务或一些日常比较常见的综合业务的测试。大数据量测试通常分为两种:①对某些存储、传输和统计查询等业务进行大数据量的测试;②与并发测试相结合的极限状态下的总和测试。主要测试运行数据量较大或历史数据量较大时 Web 应用的性能情况。

由于大数据量测试一般在投产环境下进行,因此把它独立出来和疲劳强度测试一起进行,一般在测试后期进行。

大数据量测试分实时大数据量测试和极限状态下的测试。实时大数据量测试模拟用户工作时的实时大数据量,主要目的是测试用户较多或某些业务产生大数据量时系统是否稳定;极限状态下的测试是指被测 Web 应用使用了一段时间后,数据已经积累到了一定程度时,再查看系统是否正常。

7) 配置测试

配置测试指通过测试找到系统各项资源的最优分配原则,为系统调优提供依据。配置的可变性和不稳定性是 Web 工程面临挑战的重要因素。硬件、操作系统、浏览器、存储容量、网络通信速度和多种其他客户端因素对每个用户都是难以预测的。另外,某个用户的配置可能会有规律的改变(如操作系统升级、新的 ISP 和连接速度等),可能会导致客户端环境容易出错,这些错误既微妙又重要。如果两个用户不是在相同的客户端配置工作,一个用户对 Web 应用的印象以及 Web 应用的交互方式可能与另一个用户的体验有很大不同。

配置测试不是检查每种可能的客户端配置,而是测试一组可能的客户端和服务器端配置,以确保用户在所有配置中的体验是一样的,并且将特定于特殊配置的错误分离出来。例如,可以通过不停地调整 Oracle 的内存参数来进行测试,使之达到较好的性能。配置测试本质上是和其他性能测试一起进行的。

8) 可靠性测试

软件可靠性是指在规定条件下,在规定时间内,软件无失效工作的概率。该概率受输入和使用以及其存在故障的影响,系统输入将确定是否会遇到存在的故障。而可靠性测试是指给系统增加一定业务压力的情况下,让系统运行一段时间,以此来检测系统是否稳定。

Web 应用的可靠性测试的目的如下:

(1) 通过在有使用代表性的环境中执行 Web 应用，以证实 Web 应用需求是否正确实现。

(2) 数据的准确性关系到 Web 应用可靠性评估的准确度，通过数据采集、模型选择、模型拟合以及系统可靠性评估四个步骤，为进行 Web 应用可靠性估计采集准确的数据。

(3) 找出所有对 Web 应用可靠性影响较大的错误。

2. 性能测试方法

Web 应用的性能测试需要有良好的测试方法来保证。执行 Web 应用性能测试的方法有许多，它们各有优势，其中具有代表性的主要方法有虚拟用户方法和 WUS（Website Usage Signature，站点使用签名）方法。

1) 虚拟用户方法

虚拟用户方法通过模拟真实用户的行为来对待测 Web 应用施加预期工作负载，以测量待测系统的性能，如事务的响应时间、服务器的吞吐量等。它以真实用户平时生产环境下的“事务处理”（用户为完成一个商业业务而执行的一系列操作）作为负载的基本组成单位，用“虚拟用户”（模拟用户行为的测试脚本）来模拟真实用户。工作负载的信息（如并发虚拟用户数、商务处理的执行频率等）通过人工收集和分析系统的特征来获得。

支持该方法的性能测试工具可用较少的硬件资源模拟出成百上千虚拟用户同时访问，并可模拟来自不同的地址、不同浏览器类型以及不同网络连接方式的请求，同时可实时监控相关性能指标，帮助测试人员分析测试结果。该方法易于实现和便于操作，已经为绝大多数主流的 Web 应用性能测试工具所采用，在实践中被证明是相当有效的。

这种方法需要测试人员有相当高的专业技能，需要根据预期工作负载设计准确的测试场景，以较好地模拟工作负载进行测试。

2) WUS 方法

基于 WUS 的概念来设计测试场景，强调建立真实的负载。WUS 是为了衡量测试负载和真实负载之间的接近程度，是一系列能全面刻画负载的参数和测量指标的集合，包括每小时浏览的页面、平均访问持续时间、每次访问平均浏览的页面以及页面请求分布等。这些参数可以从日志文件中得到，即需要 Web 应用的运行日志，也就是说 Web 应用正常运作一段时间后才能进行测试。参数同时也包括一些影响负载的客户端变量，如用户对站点的熟悉程度、对延迟的忍耐程度和客户端连接速度等。

此外，采用类似的 Web 应用的日志信息来模拟待测的 Web 应用工作负载方法，理论上也是可行的，只不过这个条件在现实测试环境中非常苛刻，基本上难以实现。

16.5.5 Web应用安全性测试

Web 应用运行在 Internet 上，其安全性一直广受关注，Web 应用安全性测试则是一个复杂的主题。Web 应用安全性测试是对整个 Web 应用的安全防卫措施的有效性进行测试，以揭露安全机制中的漏洞。通常的安全机制包括信息访问控制、用户身份校验以及对机密信息进行加密等，同时要能根据用户的访问情况判断出该用户是正常的用户还是蓄意的破坏者。

Web 应用的安全性测试主要包括数据加密测试、用户身份验证测试、日志文件测试、脚本

测试、Session 测试、备份与恢复测试等内容，遇到相关问题时可以根据本节所述方法进行实际操作。

1. 数据加密测试

数据加密测试是测试 Web 应用使用的关键数据是否经过了加密，所选择的加密算法是否合适等内容。

2. 用户身份验证测试

Web 应用中，用户身份验证直接关系到用户权利、隐私的安全，用户身份验证测试主要检查无效的用户名和密码能否登录，密码是否对大小写敏感，是否有验证次数的限制，是否存在不验证而直接进入 Web 应用的问题，是否存在不登录就可查看非会员页面和权限问题。

3. 日志文件测试

日志文件测试主要是检查 Web 运行的相关访问和状态信息是否写进了日志文件，是否可追踪等。此外，需测试 Web 应用执行过程中所产生的错误是否作为日志保留下来，供技术人员分析该错误是由系统实现漏洞引起的还是由于黑客攻击引起的。

日志文件测试的主要测试内容如下：

- (1) 日志是否记录所有的事务处理。
- (2) CPU 的占有率是否很高。
- (3) 是否有例外的进程占用。
- (4) 是否记录失败的注册企图。
- (5) 是否记录被盗信用卡的使用。
- (6) 是否在每次事务完成的时候都进行保存。
- (7) 是否记录 IP 地址。
- (8) 是否记录用户名等。

4. Session 测试

Session 测试主要检查 Web 应用是否有超时的限制，也就是检查用户登录 Web 应用后在一定时间内没有点击任何页面，是否需要重新登录才能正常使用，检查超时后能否自动退出，退出之后，浏览器回退按钮是否可以回到登录页面等。

5. 备份与恢复测试

根据 Web 应用对安全性的要求可以采用多种备份与恢复手段，如数据库增量备份、数据库完全备份、数据库差量备份和系统完全备份等。出于更高的安全性要求，某些实时系统经常会采用双机热备或多机热备。除了对这些备份与恢复方式进行验证测试以外，还要评估这种备份与恢复方式是否满足 Web 应用的安全性需求。

6. 访问控制策略测试

访问控制策略测试主要检查管理接口是否只有授权的管理员才允许进行访问，是否有完善的访问控制策略文档，文档中是否精确定义了每类用户可以访问的功能和内容，执行访问控制

策略的代码结构是否严谨等。

7. 安全漏洞测试

为了避免或减少 Web 应用被攻击的可能性，需要测试并发现其中隐藏的跨站脚本、命令注入、SQL 注入等安全漏洞，这类测试可借助一些安全扫描工具来实现。

8. TCP 端口测试

开放不必要的端口，会给 Web 应用带来安全隐患。所以，在部署 Web 应用之前，要用端口扫描软件对部署环境进行 TCP 端口测试，确保只开启了必要的端口。

9. 服务器端脚本漏洞检查

服务器端的脚本漏洞检查是测试存在于服务器端的脚本是否有安全漏洞，没有经过授权能否在服务器端放置和编辑脚本。

10. 防火墙测试

防火墙测试是对防火墙功能和设置进行测试，以判断是否满足 Web 应用的安全需求。

16.5.6 Web 服务测试

Web 服务在面向服务计算中得到了越来越广泛的使用，它通过在运行时动态发现和绑定服务来进行服务集成。这给 Web 服务测试，尤其是服务之间的交互测试，带来了巨大的挑战。

1. Web 服务测试特性

与传统软件测试相比，由于 Web 服务自身的特性，使其测试也具有不同特性，如表 16-1 所示。

表 16-1 Web 服务测试特性

测试因素	Web 服务测试
测试参与者	需要服务的提供者、代理和请求者共同参与到测试过程中
测试分布	分布、远程、多阶段
测试模型	不拥有服务代码（服务开发者除外），有限的测试模型（可控制性和可观测性低）
测试覆盖	除服务开发者拥有传统的覆盖范围外，服务提供者、服务集成者、服务请求者和服务代理可能只有黑盒覆盖范围
测试执行	在线、及时测试
测试客户端	需要构建测试客户端调用被测服务
测试预测	Web 服务动态绑定，很难预测其实际运行情况，难以生成测试预测
回归测试	在线、基于运行时收集数据的测试；除服务的开发者和提供者外；服务请求者、服务集成者和服务代理并不掌握服务的演化情况，回归测试需要额外的信息支持

2. Web 服务测试内容

根据 Web 服务架构和业务模型，Web 服务测试可分为三个层次：Web 服务基础设施的验证与确认、Web 服务独立测试以及 Web 服务集成测试。同时，测试组织和管理是 Web 服务测试的

三个层次中都需解决的问题，对测试的系统性、有效性至关重要。

1) Web 服务基础设施的验证与确认

虽然标准化和开放性是 Web 服务的主要宗旨，但与传统的应用系统相比，Web 服务中间件的稳定性和可靠性相对薄弱。一方面，是由于 Web 服务的整个技术架构还尚未成为成熟的产业标准。在 W3C 联盟以及相关研究机构的大力支持下，标准规范体系还处于不断发展和完善的过程中，不同的标准之间的概念模型和表示体系可能互不兼容，并进一步导致相关应用程序的不兼容。另一方面，Web 服务规范体系采用 XMI 作为基本的编码语言。由于 XMI 技术的简单、灵活、可伸缩、易定制，基于 XML 数据描述和 XMI Schema 数据建模定义的 Web 服务的协议栈具有标准化和开放性的特点。同时，XMI 为描述性语言，其通用性和灵活性为协议的可证明性提出了挑战。当 Web 服务用于高可信性或实时要求较高的应用中时，需要更严格的验证方法。例如，SOAP RPC 是 Web 服务的基础和核心协议，由于编码格式的不同，SOAP RPC 与传统的 DCE RPC 及 ONC RPC 采用了完全不同的消息映射和处理机制。SOAP 还处于完善过程中，对于 SOAP 本身的验证还有待进一步加强。

2) Web 服务独立测试

Web 服务首先作为独立的功能节点发布，再通过 workflow 定义和解析动态集成为完整的业务流程。Web 服务独立测试从以下三方面保证各服务节点的质量：①服务的实现应在功能、性能等各方面与发布的服务描述相一致，为验证一致性，除服务提供者外，服务代理及用户都应在一定的安全约束下，远程测试该服务；②由于服务发布的开放性，对于每个服务请求，可能存在多个满足需求的服务描述，服务代理应根据一定的度量和评价标准，对多个服务进行测试、比较和评估，并依照需求的满足程度排序；③在服务实现的演化过程中，应建立一定的机制来支持对不同版本的跟踪及回归测试。

3) Web 服务集成测试

通过对服务流描述的解析，Web 服务可以动态地集成。Web 服务集成的描述、解析和执行将是 Web 服务区别于其他分布式计算技术的主要特征。集成的 Web 服务测试就是在服务流描述执行前，通过静态验证以及动态模拟的方法，确认服务描述能够正确地描述业务需求，能够由服务中介正确解析，并能由所有服务节点正确执行。